Predicting Student Dropout and Academic Success

A Supervised Machine Learning Comparison of Logistic Regression and Random Forest

Fundamentals of Machine Learning

	Member 1	**Member 2**
Name	Thuiyadura L.R	Nissanka H.M.D.P
Student ID	MS26923956	MS26930190
Algorithm	Logistic Regression	Random Forest
Notebook	logistic_regression.ipynb	random_forest.ipyn
Dataset	https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success	
GitHub repository	https://github.com/LoshithaAI/ML-Assignment.git	
Presentation video	https://youtu.be/X3Lj_QzbAp0?si=mFC33z6V3WgRJBcm	

Abstract

Higher-education institutions need to identify students who are likely to drop out early enough to help them. This report addresses that problem as a three-class classification task (Dropout, Enrolled, Graduate) on the public UCI dataset Predict Students' Dropout and Academic Success (4,424 students, 36 attributes). Two distinct supervised algorithms were applied to exactly the same stratified 80/20 train/test split: a regularised multinomial Logistic Regression (a linear, interpretable model) and a Random Forest (a non-linear ensemble of decision trees). Both were tuned by 5-fold stratified cross-validation on the training data using macro-F1 as the selection metric. On the held-out test set Random Forest reached 75.9% accuracy, macro-F1 0.703 and macro-AUC 0.887, while Logistic Regression reached 73.4% accuracy, macro-F1 0.696 and macro-AUC 0.884. The accuracy gap (+2.5 points) is borderline (McNemar $p = 0.055$) and the macro-F1 and AUC differences are within noise. Logistic Regression finds more of the hard "Enrolled" students (recall 0.635 vs 0.509), and its coefficients show that tuition-fee status and approved curricular units are the strongest protective factors. Almost all predictive power comes from academic results achieved after enrolment.

Contents

Abstract 2
1.Introduction and problem statement 6
2. Dataset 6
2.1 Source and context 6
2.2 Why this dataset is appropriate 6
2.3 Attributes 7
2.4 Class distribution 7
3.Data preprocessing 8
3.1 Data cleansing 8
3.2 Outlier analysis 8
3.3 Exploratory data analysis 9
3.4 Encoding categorical features 10
3.5 Train/test split 10
3.7 Dimensionality reduction 11
4. Methodology: algorithms and justification 11
4.1 Logistic Regression (Member 1: Thuiyadura L.R) 11
4.2 Random Forest (Member 2: Nissanka H.M.D.P) 12
5. Implementation 13
6. Results 14
6.1 Logistic Regression 14
Discussion of the Logistic Regression results 17
Model interpretation 18
6.2 Random Forest 19
Model interpretation 22
7. Comparison of the two algorithms 23
7.1 Design differences 23
7.2 Test-set accuracy and related metrics 24
7.3 Is the difference meaningful? 24
7.4 Discussion of the comparison 25
8. Critical analysis and discussion 26
8.1 Improvement and ablation experiments 26
8.2 Learning curves 27

8.3 Why accuracy is limited 29
8.4 How the accuracy could be improved 29
9. Limitations and future work 29
9.1 Limitations 29
9.2 Future work 30
10. Individual contributions 30
Shared work (both members) 30
Individual work 30
11. Conclusion 31
12. References 31
Appendix A – Source code: Logistic Regression notebook (logistic_regression.ipynb) 32
Appendix B – Source code: Random Forest notebook (random_forest.ipynb) 50

Figure 1 Class distribution. Always predicting "Graduate" would give only ~50% accuracy, which is the majority-class baseline. 8
Figure 2 Box plots of age at enrolment, admission grade and 1st-semester grade, used for outlier inspection. 9
Figure 3 Approved units (1st and 2nd semester) and admission grade by outcome. 9
Figure 4 Correlation matrix of the continuous features. 10
Figure 5 Cumulative explained variance of the principal components (encoded training data). 11
Figure 6 Effect of C on training and validation macro-F1. 15
Figure 7 Logistic Regression (tuned) classification report on the 885 test students. 15
Figure 8 Logistic Regression confusion matrices (counts and row-normalised). 16
Figure 9 Logistic Regression one-vs-rest ROC curves 16
Figure 10 Baseline vs. tuned Logistic Regression 17
Figure 11 Model interpretation 18
Figure 12 Top Random Forest settings from 5-fold cross-validation (training data only). 19
Figure 13 Train vs. validation macro-F1 for all 20 tried settings 19
Figure 14 Random Forest (tuned) classification report on the 885 test students. Macro-AUC (one-vs-rest) = 0.887; 5-fold CV accuracy 0.771 ± 0.015, CV macro-F1 0.717 ± 0.020; OOB accuracy 0.769. 20
Figure 15 Random Forest confusion matrices (counts and row-normalised). 20
Figure 16 Random Forest one-vs-rest ROC curves. 21
Figure 17 Baseline vs. tuned Random Forest. 21
Figure 18 Permutation importance on the test set (drop in macro-F1 when a feature is shuffled). 22
Figure 19 Permutation importance on the test set (drop in macro-F1 when a feature is shuffled) 23
Figure 20 – Logistic Regression improvement / ablation experiments (test set). 26
Figure 21 Random Forest improvement / ablation experiments (test set). 27

Figure 22 – Logistic Regression learning curve. 28
Figure 23 Random Forest learning curve 28

Table 1 The 36 input attributes grouped by theme. The target is Target ∈ {Dropout, Enrolled, Graduate}. 7
Table 2 Distribution of the target variable. 7
Table 3 Logistic Regression hyperparameter grid (12 combinations, GridSearchCV, 5-fold stratified CV, scoring = macro-F1). 12
Table 4 Random Forest search space (RandomizedSearchCV, 20 random combinations, 5-fold stratified CV, scoring = macro-F1). 13
Table 5 Top Logistic Regression settings from 5-fold cross-validation (training data only). 14
Table 6 How the two algorithms differ in design and use. 24
Table 7 Statistical comparison of the two models. 25
Table 8 – Logistic Regression experiments 26
Table 9 Random Forest experiments. 27

1.Introduction and problem statement

Student dropout is costly for students, who lose time and money, and for institutions, which lose revenue and fail their social mission. If the students most at risk can be flagged early, the institution can offer tutoring, financial support or counselling. The task addressed here is: given the information an institution holds about a student, predict whether the student will drop out, remain enrolled, or graduate at the end of the normal course duration. The assignment requires a real-world, publicly published dataset and two distinct supervised algorithms, compared in terms of accuracy using appropriate metrics. The team therefore used one shared dataset and one shared experimental protocol, and each member implemented one algorithm. The two algorithms were chosen to contrast each other deliberately: a linear, interpretable model (Logistic Regression) against a non-linear tree ensemble (Random Forest). Only classical (non-deep-learning) machine learning is used, as required. The report covers the dataset and its preprocessing, the background and justification of each algorithm, the implementation, test results, a head-to-head comparison (including statistical checks), a critical analysis of how accuracy could be improved, limitations, future work, and each member's contribution. The full source code of both notebooks is included as text in the appendices.

2. Dataset

2.1 Source and context

Dataset: Predict Students' Dropout and Academic Success, UCI Machine Learning Repository, https://archive.ics.uci.edu/dataset/697 (licence CC BY 4.0). The copy used for the experiments is stored in the project repository (https://github.com/LoshithaAI/ML-Assignment, file data/data.csv, semicolon-separated). Citation: Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success

The data come from a Portuguese higher-education institution and combine information known at enrolment (demographics, socioeconomic background, academic path), the student's academic performance at the end of the 1st and 2nd semesters, and national macroeconomic indicators. Each row is one student on an undergraduate programme (e.g. agronomy, design, education, nursing, journalism, management, social service, technologies). It was created to identify students at risk of dropping out.

2.2 Why this dataset is appropriate

• It is real-world data published on a credible public repository (UCI) and is not a standard tutorial dataset such as Iris or Titanic.

• It is reasonably complex: 4,424 rows, 36 features and 3 classes, with mixed feature types (categorical codes, binary flags, continuous grades).

• The classes are imbalanced ("Enrolled" is only ~18%), which makes evaluation with the right metrics important.

2.3 Attributes

Group	Attributes	Type
Demographic	Marital status, Nacionality, Gender, Age at enrollment, International, Displaced	Categorical code / binary / numeric
Application	Application mode, Application order, Course, Daytime/evening attendance	Categorical code / ordinal / binary
Previous education	Previous qualification, Previous qualification (grade), Admission grade	Categorical code / numeric
Family background	Mother's and Father's qualification and occupation	Categorical code
Financial / special	Educational special needs, Debtor, Tuition fees up to date, Scholarship holder	Binary
1st-semester performance	Curricular units 1st sem: credited, enrolled, evaluations, approved, grade, without evaluations	Numeric
2nd-semester performance	Curricular units 2nd sem: credited, enrolled, evaluations, approved, grade, without evaluations	Numeric
Macroeconomic	Unemployment rate, Inflation rate, GDP	Numeric

Table 1 The 36 input attributes grouped by theme. The target is Target ∈ {Dropout, Enrolled, Graduate}.

Nine attributes (Marital status, Application mode, Course, Previous qualification, Nacionality, and the parents' qualification and occupation) hold integer codes for categories, not quantities. In total the table has 9 categorical-code columns, 8 binary columns and 18 continuous columns (plus Application order, treated as ordinal).

2.4 Class distribution

Class	Students	Share
Graduate	2,209	49.9%
Dropout	1,421	32.1%
Enrolled	794	17.9%

Table 2 Distribution of the target variable.

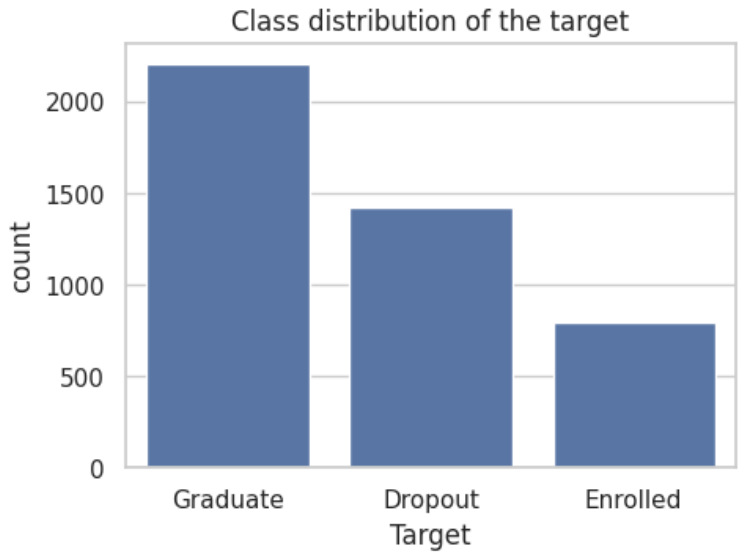


Figure 1 Class distribution. Always predicting "Graduate" would give only ~50% accuracy, which is the majority-class baseline.

Selected summary statistics of the continuous features: age at enrolment has mean 23.3 (range 17–70); admission grade has mean 127.0 (95–190); approved units average 4.7 in the 1st semester and 4.4 in the 2nd; the unemployment rate averages 11.6%.

3.Data preprocessing

Both members applied the same preprocessing decisions so that the models are compared on identical inputs. Model-specific steps are noted explicitly.

3.1 Data cleansing

There are no missing values and no duplicate rows, so no imputation or row removal was needed. The only cleaning required was to read the file with the semicolon delimiter and strip whitespace from column names. All columns are numeric except the target.

3.2 Outlier analysis

The IQR rule (values outside [Q1 − 1.5·IQR, Q3 + 1.5·IQR]) flagged many values, mainly in 2nd-semester grade (19.8%), 1st-semester grade (16.4%), credited units (12–13%) and age at enrolment (10.0%).

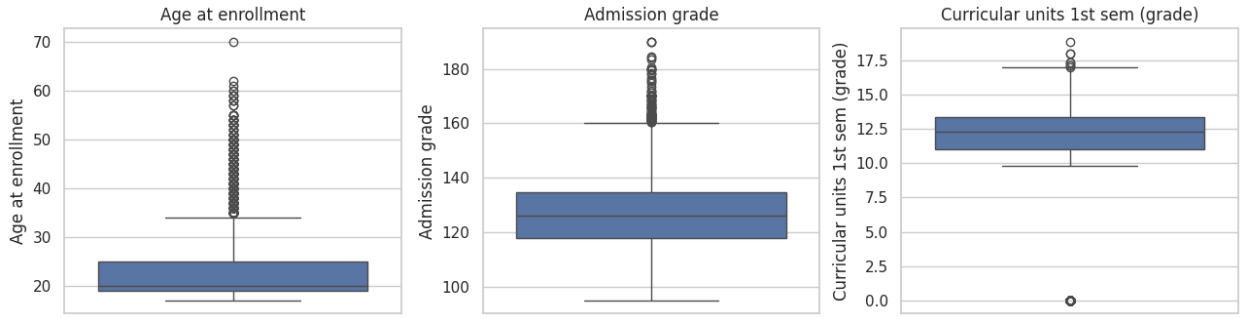


Figure 2 Box plots of age at enrolment, admission grade and 1st-semester grade, used for outlier inspection.

Decision: all flagged values were kept. They are genuine observations (mature students, students who passed no units and therefore have a grade of zero), not data-entry errors, and may carry exactly the signal needed to identify at-risk students. Random Forest is robust to outliers because splits depend on ordering, not magnitude. For Logistic Regression, standardisation does not remove outliers, but L2 regularisation limits their influence on the coefficients.

3.3 Exploratory data analysis

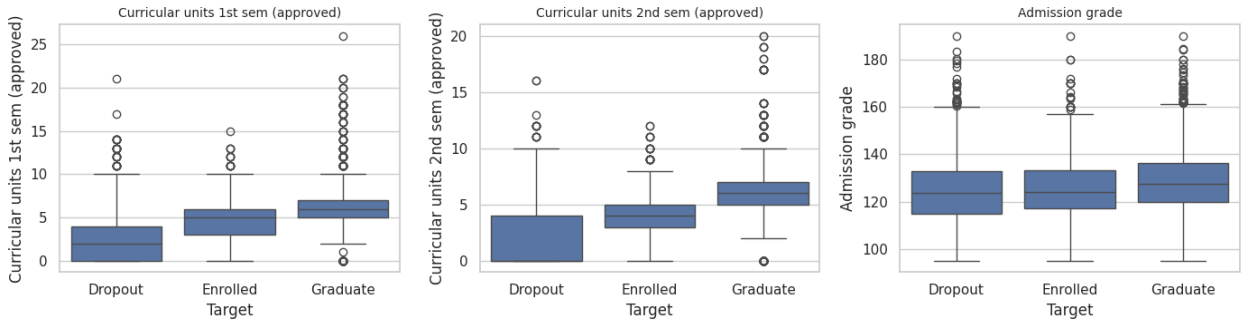


Figure 3 Approved units (1st and 2nd semester) and admission grade by outcome.

• **Approved units separate the classes clearly.** Median approved units are 2 (1st sem.) and 0 (2nd sem.) for Dropout, 5 and 4 for Enrolled, and 6 and 6 for Graduate. Dropout and Enrolled still overlap.

• **Admission grade is a weak separator on its own:** medians are about 124, 124 and 127 for Dropout, Enrolled and Graduate.

• "Enrolled" sits between the other two classes on almost every feature, which suggests it will be the hardest class to predict.

• Features from the two semesters are strongly correlated (0.78–0.94 for matching pairs), so they carry partly redundant information (multicollinearity).

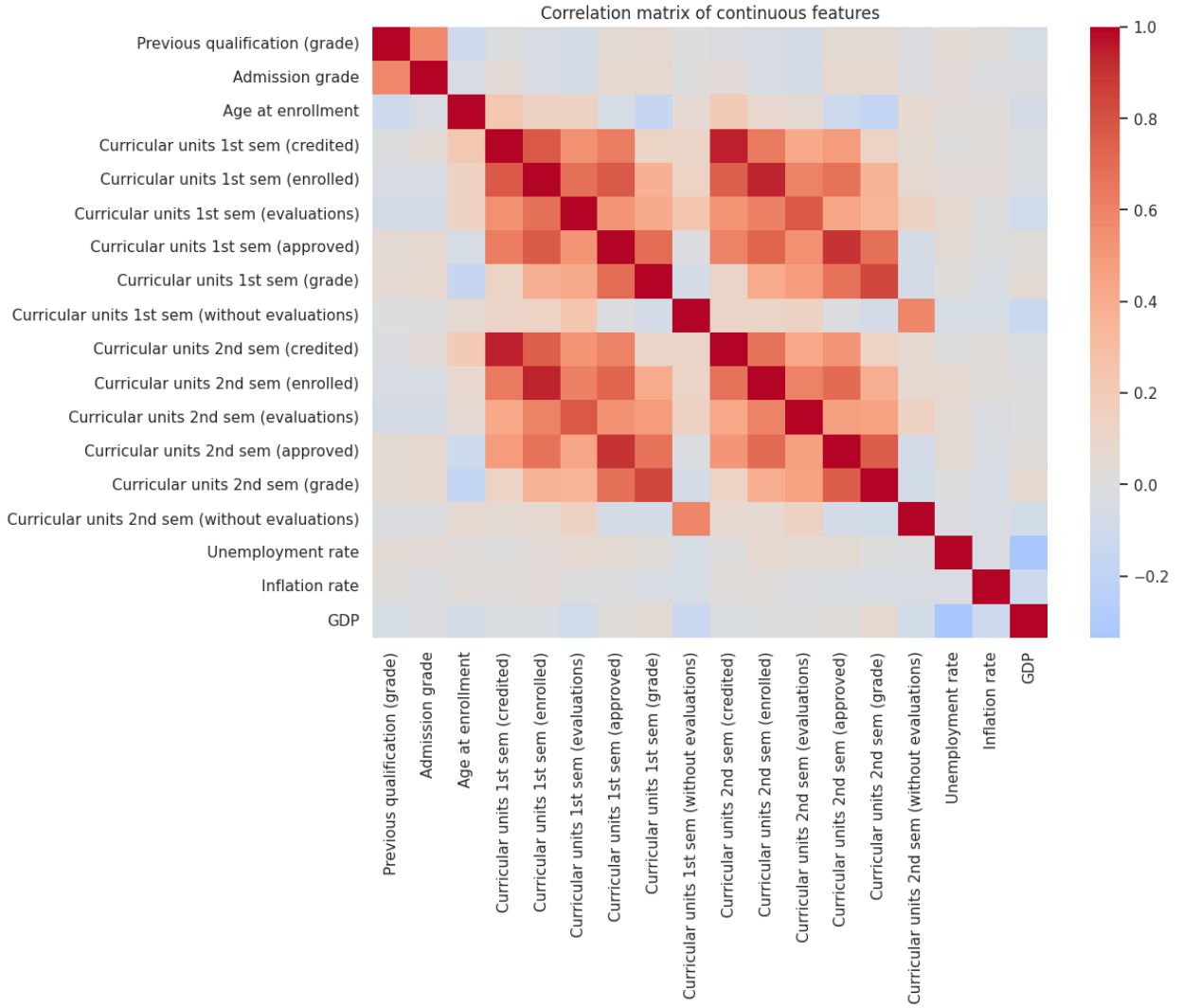


Figure 4 Correlation matrix of the continuous features.

3.4 Encoding categorical features

The nine code-valued columns are labels, not quantities: a linear model would wrongly treat Course = 9254 as "larger" than Course = 171. They were one-hot encoded with drop_first=True (one dummy removed per feature to avoid perfect collinearity). Binary and continuous columns were left unchanged. This expands the 36 features to 238 columns.

3.5 Train/test split

The data were split 80/20 with stratification on the target and random_state = 42, giving 3,539 training and 885 test students. Class proportions are preserved in both parts (train: Graduate 49.9%, Dropout 32.1%, Enrolled 17.9%; test: 49.9%, 32.1%, 18.0%). The test set was used only once for the final evaluation of each tuned model; all tuning used cross-validation on the training set only. 3.6 Feature scaling Logistic Regression: the 18 continuous columns were standardised (mean 0, standard deviation 1) inside a scikit-learn Pipeline, so the scaler is fitted only on the training part

of each cross-validation fold and never sees validation or test data (no data leakage). Random Forest: no scaling is needed because each split only asks whether a feature is above or below a threshold, which depends on order rather than scale.

3.7 Dimensionality reduction

After encoding there are 238 columns. For Logistic Regression a PCA analysis showed that 48 components keep 95% of the variance (about 80% fewer dimensions).

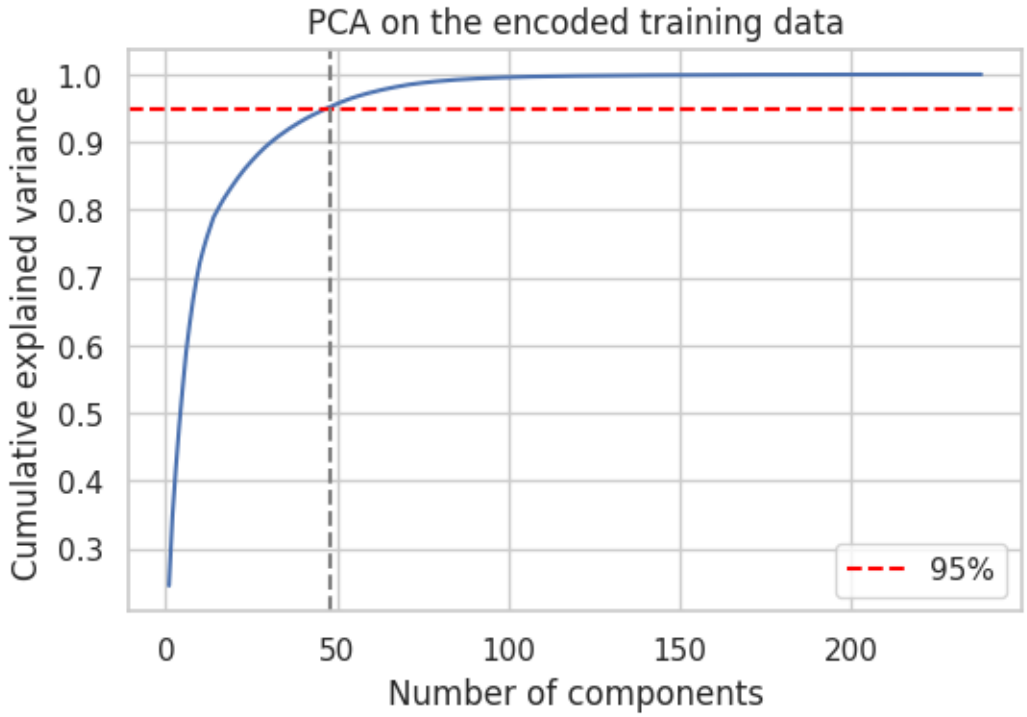


Figure 5 Cumulative explained variance of the principal components (encoded training data).

PCA was not used in the main Logistic Regression model, because each component mixes many original features and would remove the interpretability that is the main strength of the model; its effect on accuracy was tested as an experiment (Section 8, E3) and L1 regularisation was tested as an alternative form of feature selection (E2). For Random Forest, feature selection by importance was used instead of PCA (Section 8, E4).

4. Methodology: algorithms and justification

4.1 Logistic Regression (Member 1: Thuiyadura L.R)

Background. Logistic Regression is a supervised, linear classification algorithm. For a student with features x it computes a score for each class k as a linear combination $z_k = w_k \cdot x + b_k$. For more than two classes the softmax function turns the scores into probabilities that sum to 1: $P(k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j)$. The student is assigned to the class with the highest probability.

Learning. The weights minimise the cross-entropy (log-loss) between predicted probabilities and true labels plus a regularisation penalty. The strength of the penalty is controlled by C, the inverse of the regularisation strength (small C = strong regularisation). L2 shrinks all weights; L1 drives some weights to exactly zero (built-in feature selection). The problem is convex, so the optimum is global; the lbfgs solver was used.

Why it suits this problem.

• The task is multi-class classification with labelled data, which multinomial Logistic Regression supports directly.

• It outputs probabilities, useful for ranking students by dropout risk.

• It is interpretable: signed coefficients show which factors push a student towards dropout or graduation, valuable for planning interventions.

• It is a fast, strong baseline, and regularisation copes with the many correlated and one-hot features.

• class_weight="balanced" can address the class imbalance.

• It contrasts clearly with Random Forest (linear vs. non-linear), making the comparison meaningful.

Assumptions and limitations. It assumes a linear relationship between the features and the log-odds of each class, needs scaled and encoded inputs, and cannot capture interactions or non-linear patterns unless they are engineered manually.

Hyperparameter	Meaning	Values tried
C	Inverse regularisation strength	0.001, 0.01, 0.1, 1, 10, 100
class_weight	Penalise mistakes on small classes more	None, "balanced"

Table 3 Logistic Regression hyperparameter grid (12 combinations, GridSearchCV, 5-fold stratified CV, scoring = macro-F1).

4.2 Random Forest (Member 2: Nissanka H.M.D.P)

Background. A Random Forest is a supervised ensemble that builds many decision trees and combines their votes. A decision tree repeatedly splits the data with yes/no questions (for example "approved units in the 2nd semester ≤ 4?"), choosing at each node the split that makes the resulting groups as pure as possible, measured by Gini impurity (Gini = 1 − Σ pk 2). A single deep tree fits the training data very closely and overfits (high variance).

A Random Forest reduces this variance with two sources of randomness: bagging (each tree is trained on a bootstrap sample of the rows) and random feature subsets (only max_features features are considered at each split, which de-correlates the trees). The final prediction is the majority vote or the average of class probabilities. The rows left out of each bootstrap sample give a free out-of-bag (OOB) accuracy estimate.

Why it suits this problem.

• It handles mixed feature types and non-linear relationships and interactions (e.g. scholarship × debt × grades) without manual feature engineering.

• It needs no feature scaling and is robust to outliers.

• It copes well with many (238) and correlated features, and supports class weighting for the minority class.

• It provides feature importance, so the institution can see which factors drive dropout.

• It contrasts clearly with Logistic Regression, making the comparison meaningful.

Limitations. It is less interpretable than a linear model, slower to train and predict, impurity-based importances are biased towards features with many distinct values, and it can still overfit noisy data if trees are left unrestricted.

Hyperparameter	Meaning	Values searched
n_estimators	Number of trees	100, 200, 300, 500
max_depth	Maximum depth of each tree	None, 8, 12, 16, 24
min_samples_leaf	Minimum samples in a leaf	1, 2, 4, 8
max_features	Features considered per split	"sqrt", "log2", 0.3
class_weight	Penalise mistakes on small classes more	None,"balanced", "balanced_subsample"

Table 4 Random Forest search space (RandomizedSearchCV, 20 random combinations, 5-fold stratified CV, scoring = macro-F1).

Metric choice. For both models the tuning metric is macro-F1 (the unweighted mean of the per-class F1 scores), because accuracy alone hides poor performance on the small "Enrolled" class.

5. Implementation

Both notebooks are written in Python in Jupyter Notebook using pandas 2.2.3, NumPy 2.1.3, scikit-learn 1.6.1, matplotlib and seaborn (Python 3.13). Each notebook is commented step by step and follows the same structure: setup, dataset, preprocessing, algorithm background, baseline model, hyperparameter tuning, test evaluation, interpretation, improvement experiments and discussion. The complete code is in the appendices and in the GitHub repository (https://github.com/LoshithaAI/ML-Assignment), which has one branch per member.

• **Reproducibility:** a fixed seed (RANDOM_STATE = 42) is used for the split, the cross-validation folds and the models, and the two members use identical split settings.

• **No leakage:** for Logistic Regression, scaling and the classifier are chained in a Pipeline so that scaling is fitted inside each CV fold.

• **Baselines:** each algorithm is first run with default settings, to measure what tuning adds.

• **Evaluation:** accuracy, per-class precision/recall/F1, macro-averaged metrics, confusion matrices (counts and row-normalised), one-vs-rest ROC curves and AUC, and cross-validation on the training data.

• **Results exchange:** each notebook saves its metrics to a JSON file (logistic_regression_results.json, random_forest_results.json) so that the comparison uses the stored numbers.

6. Results

6.1 Logistic Regression

Tuning. The grid search selected C = 0.1 with class_weight = "balanced" (best cross-validated macro-F1 0.715 ± 0.015).

C	class_weight	Mean train macro-F1	Mean validation macro-F1	Std (validation)
0.1	balanced	0.749	0.715	0.015
1	balanced	0.765	0.710	0.014
10	balanced	0.772	0.704	0.015
100	balanced	0.772	0.701	0.017
1	None	0.750	0.697	0.015
0.01	balanced	0.713	0.693	0.015

Table 5 Top Logistic Regression settings from 5-fold cross-validation (training data only).

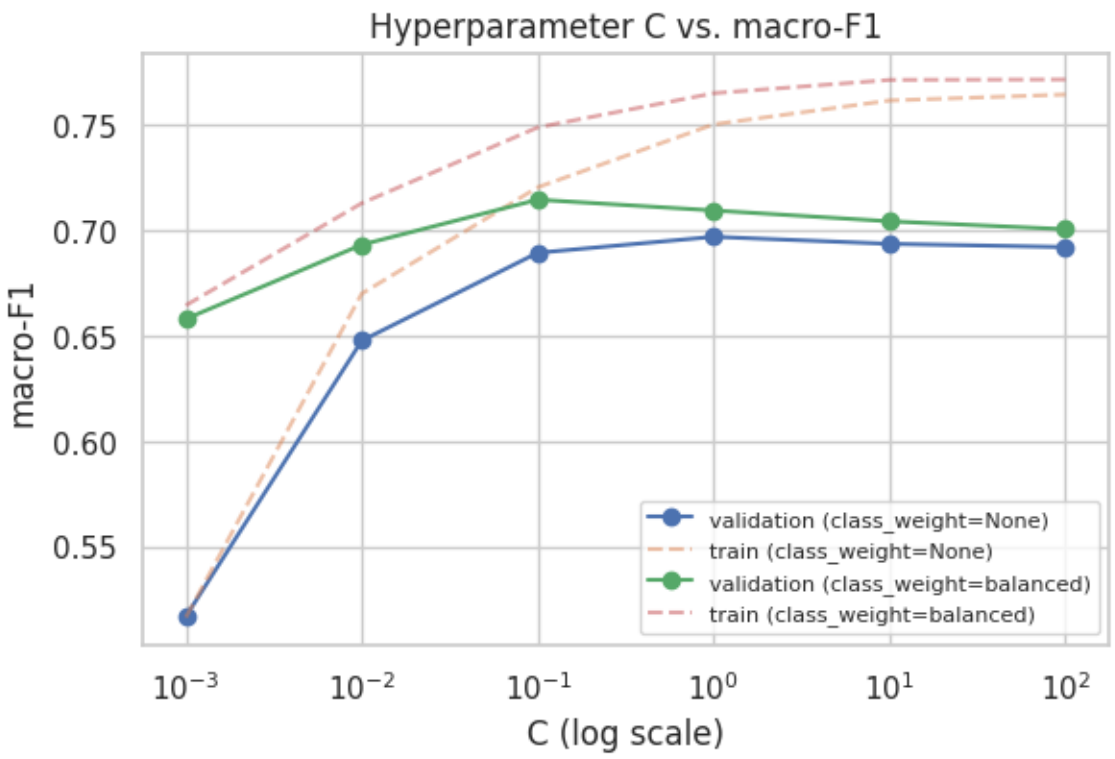


Figure 6 Effect of C on training and validation macro-F1.

Reading the plot. Validation macro-F1 peaks at C = 0.1. With very strong regularisation (C = 0.01) both training (0.713) and validation (0.693) scores are lower, indicating underfitting. As C grows beyond 0.1 the training score rises (to 0.772) while validation falls (to 0.701), so the train–validation gap widens from 0.034 to 0.071, a sign of mild overfitting. Balanced class weights beat no weighting at the same C (0.710 vs 0.697 at C = 1). The whole range is narrow (0.69–0.715) and the fold-to-fold standard deviation is about 0.015, so the model is not very sensitive to C.

Test-set performance (evaluated once with the final tuned model):

```
Test accuracy : 0.734
Test macro-F1 : 0.696

              precision    recall  f1-score   support

     Dropout      0.824     0.690     0.751       284
    Enrolled      0.417     0.635     0.504       159
    Graduate      0.872     0.799     0.834       442

    accuracy                          0.734       885
   macro avg      0.704     0.708     0.696       885
weighted avg      0.775     0.734     0.748       885
```

Figure 7 Logistic Regression (tuned) classification report on the 885 test students.

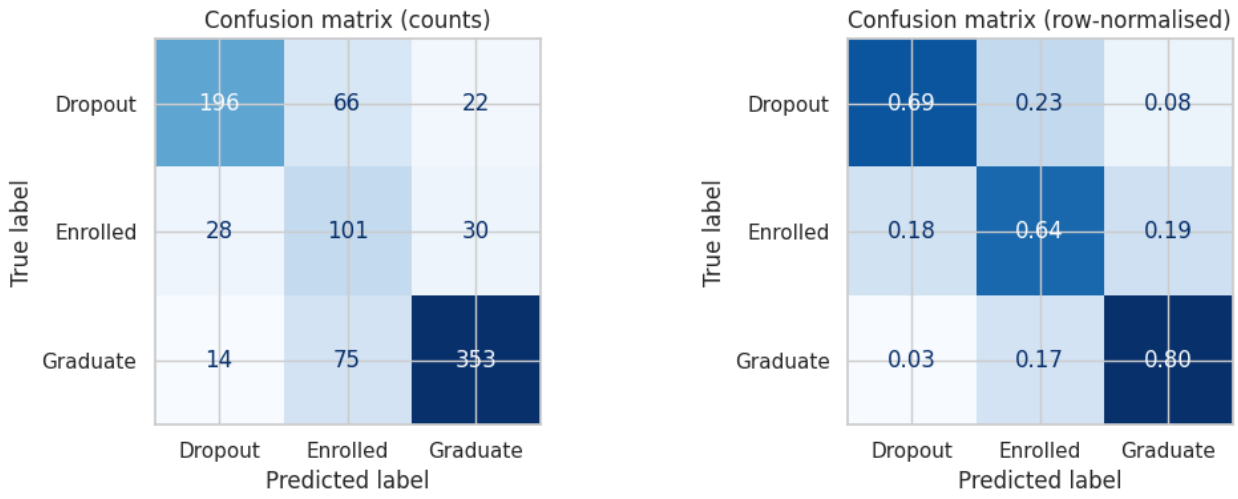


Figure 8 Logistic Regression confusion matrices (counts and row-normalised).

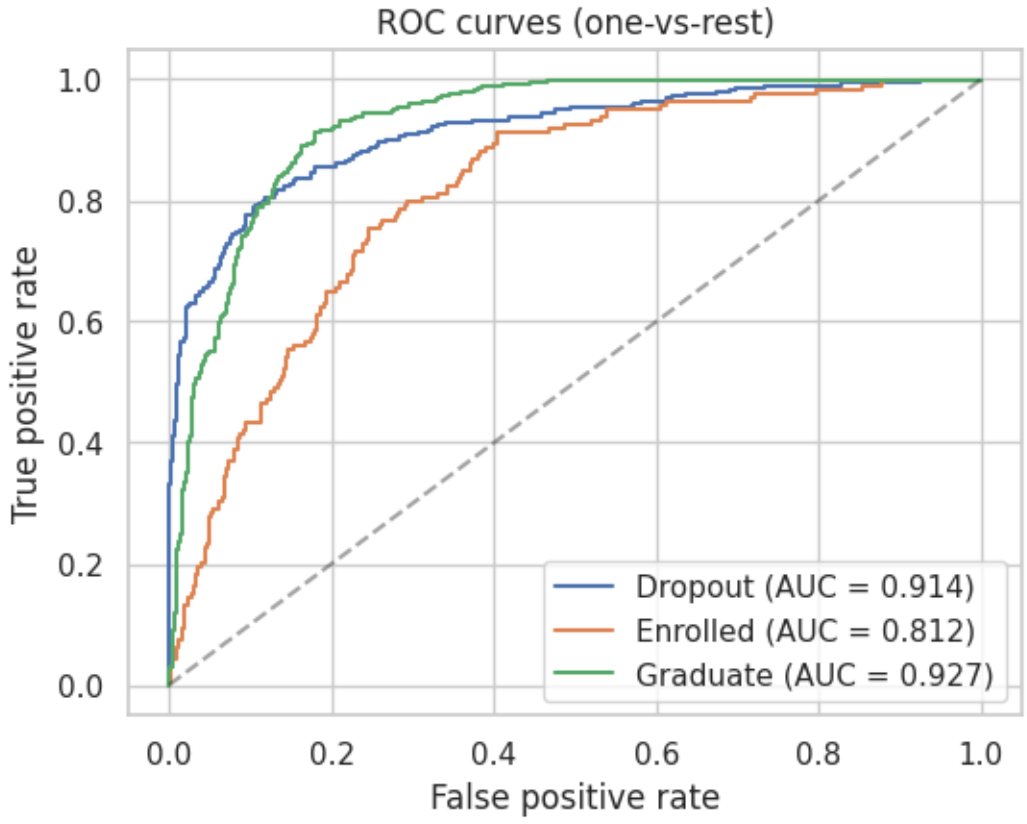


Figure 9 Logistic Regression one-vs-rest ROC curves

	Model	Accuracy	Macro-F1	Macro-Precision	Macro-Recall
0	Logistic Regression (baseline)	0.773	0.696	0.719	0.687
1	Logistic Regression (tuned)	0.734	0.696	0.704	0.708

Figure 10 Baseline vs. tuned Logistic Regression

Discussion of the Logistic Regression results

Overall: 73.4% accuracy, macro-F1 0.696 and macro-AUC 0.884, far above the ~50% majority-class baseline, so the model learns real signal.

• **Per class:** Graduate is predicted best (F1 0.834), then Dropout (0.751). Enrolled is hardest (F1 0.504, precision 0.417, recall 0.635): these students sit between the other two groups.

• **Confusion matrix:** 101 of 159 Enrolled students are found, but 75 Graduates and 66 Dropouts are also labelled Enrolled. This is why Enrolled precision is low: balanced weights push the model to predict the small class more often.

• **Consistency:** test macro-F1 (0.696) is close to the cross-validated value (0.715 ± 0.015), so the model generalises and shows no sign of serious overfitting.

• **Baseline vs. tuned:** the same macro-F1 (0.696). The baseline has higher accuracy (0.773) because it rarely predicts "Enrolled"; the tuned model gives up accuracy for much better Enrolled recall. An institution that wants to find students needing attention would accept the extra false alarms.

Model interpretation

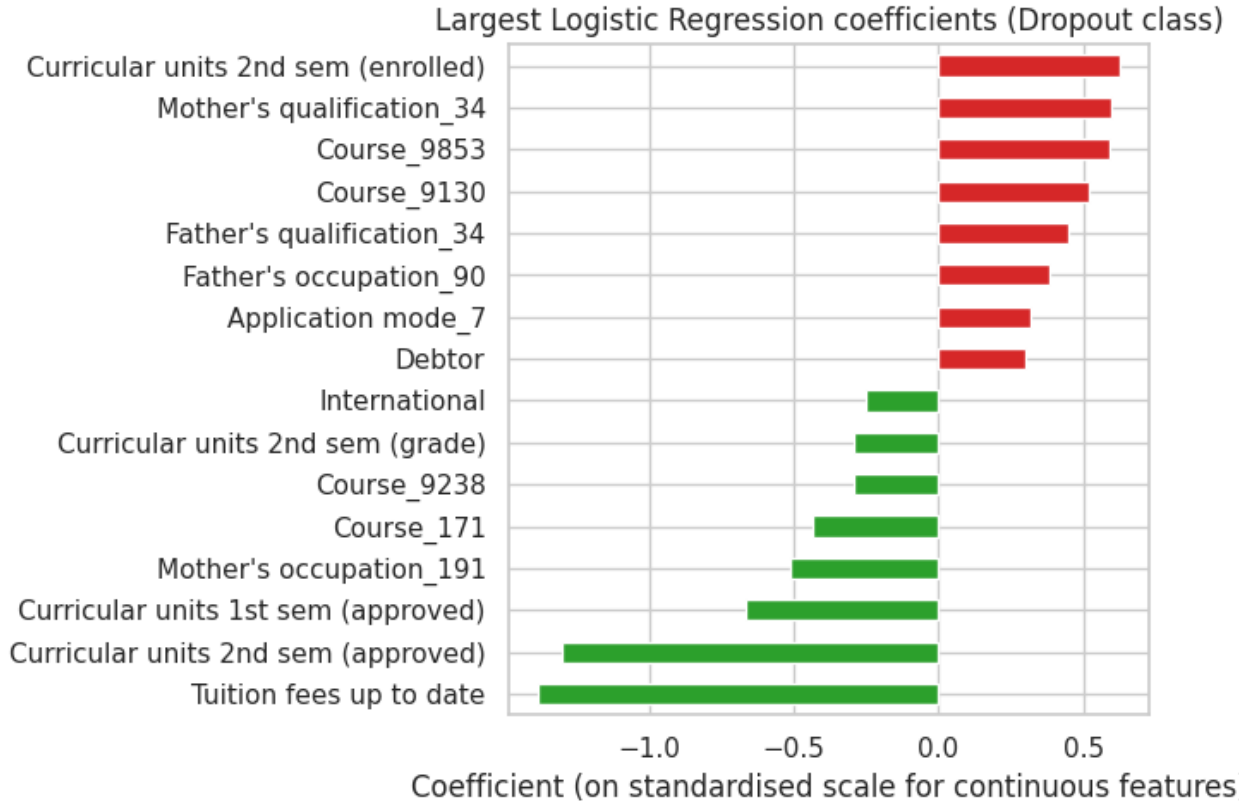


Figure 11 Model interpretation

Protective factors: up-to-date tuition fees and approved units in both semesters. Risk factor: being a debtor (financial stress).

• Some large positive coefficients belong to specific course or parent-qualification categories, several of which are small groups (e.g. Mother's occupation 191 has only 26 students), so those coefficients are less reliable.

• **A counter-intuitive sign:** 2nd-semester enrolled units has a positive coefficient (+0.63). A likely explanation is multicollinearity (enrolled and approved units are correlated, r = 0.70): with approved units already in the model, more enrolled units means a lower pass rate, which signals risk. Single coefficients should therefore be read with care.

• Coefficients show association, not causation, and depend on the reference category dropped in the one-hot encoding.

6.2 Random Forest

Tuning. The randomised search selected 300 trees, max_depth = 16, min_samples_leaf = 2, max_features = "sqrt", class_weight = "balanced" (best cross-validated macro-F1 0.717).

	param_n_estimators	param_max_depth	param_min_samples_leaf	param_max_features	param_class_weight	mean_train_score	mean_test_score
9	300	16	2	sqrt	balanced	0.923	0.717
14	300	16	8	0.3	balanced	0.825	0.713
15	100	8	4	0.3	balanced	0.799	0.712
0	100	12	2	sqrt	balanced	0.861	0.712
4	200	24	2	log2	balanced	0.882	0.707
17	500	16	1	sqrt	balanced_subsample	0.980	0.705
1	300	8	1	sqrt	balanced	0.788	0.704
8	100	8	2	sqrt	balanced	0.771	0.704

Figure 12 Top Random Forest settings from 5-fold cross-validation (training data only).

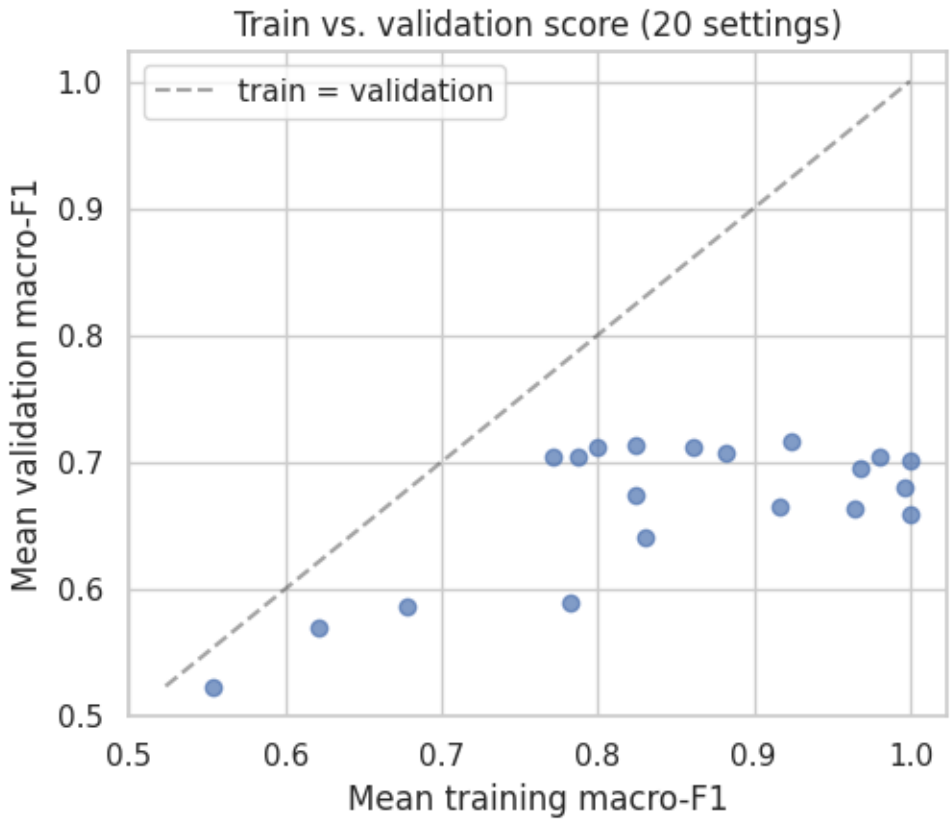


Figure 13 Train vs. validation macro-F1 for all 20 tried settings

Reading the plot. Every point lies below the dashed line, so all 20 settings score higher on the training folds than on validation folds, i.e. some overfitting is present in all of them. The gap is largest for deep, unrestricted trees (min_samples_leaf = 1: training macro-F1 0.980 but validation 0.705). Shallower or more restricted settings (e.g. max_depth = 8) have much smaller gaps but similar validation scores (0.704–0.712). The best setting still has a training score of 0.923, so a

gap remains. Tuning improved validation performance only slightly (top settings differ by about 0.01); the main gains came from class weights and limiting depth and leaf size.

Test-set performance (evaluated once with the final tuned model):

```
Test accuracy : 0.759
Test macro-F1 : 0.703

              precision    recall  f1-score   support

     Dropout      0.812     0.715     0.760       284
    Enrolled      0.474     0.509     0.491       159
    Graduate      0.836     0.878     0.857       442

    accuracy                          0.759       885
   macro avg      0.707     0.701     0.703       885
weighted avg      0.763     0.759     0.760       885
```

Figure 14 Random Forest (tuned) classification report on the 885 test students. Macro-AUC (one-vs-rest) = 0.887; 5-fold CV accuracy 0.771 ± 0.015, CV macro-F1 0.717 ± 0.020; OOB accuracy 0.769.

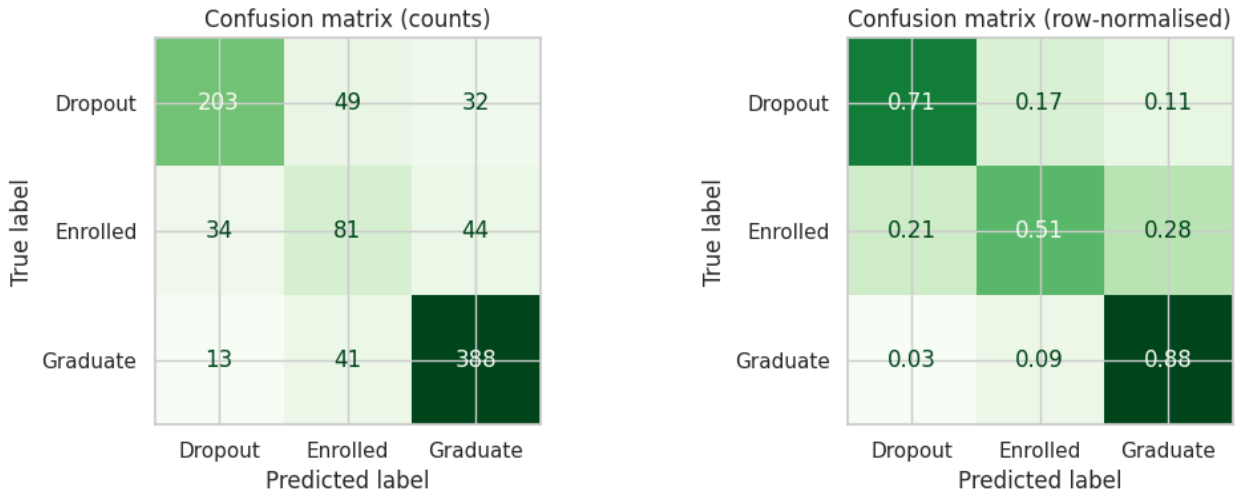


Figure 15 Random Forest confusion matrices (counts and row-normalized).

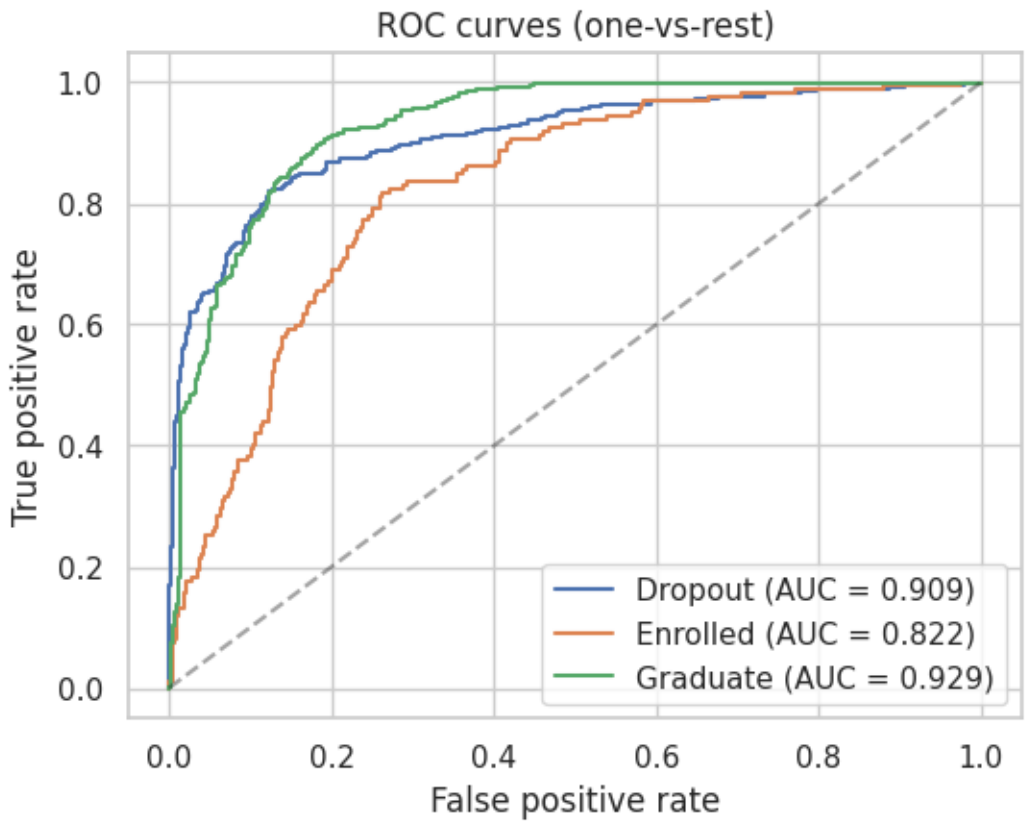


Figure 16 Random Forest one-vs-rest ROC curves.

	Model	Accuracy	Macro-F1	Macro-Precision	Macro-Recall
0	Random Forest (baseline)	0.766	0.668	0.718	0.660
1	Random Forest (tuned)	0.759	0.703	0.707	0.701

Figure 17 Baseline vs. tuned Random Forest.

The baseline forest has slightly higher accuracy (0.766 vs 0.759) and macro-precision, but the tuned forest is better on macro-F1 (0.703 vs 0.668) and macro-recall (0.701 vs 0.660). The baseline reaches its accuracy mostly by predicting the large Graduate and Dropout classes well, with weak recall on “Enrolled”. Because macro metrics treat each class equally, the tuned model is the better choice.

Model interpretation

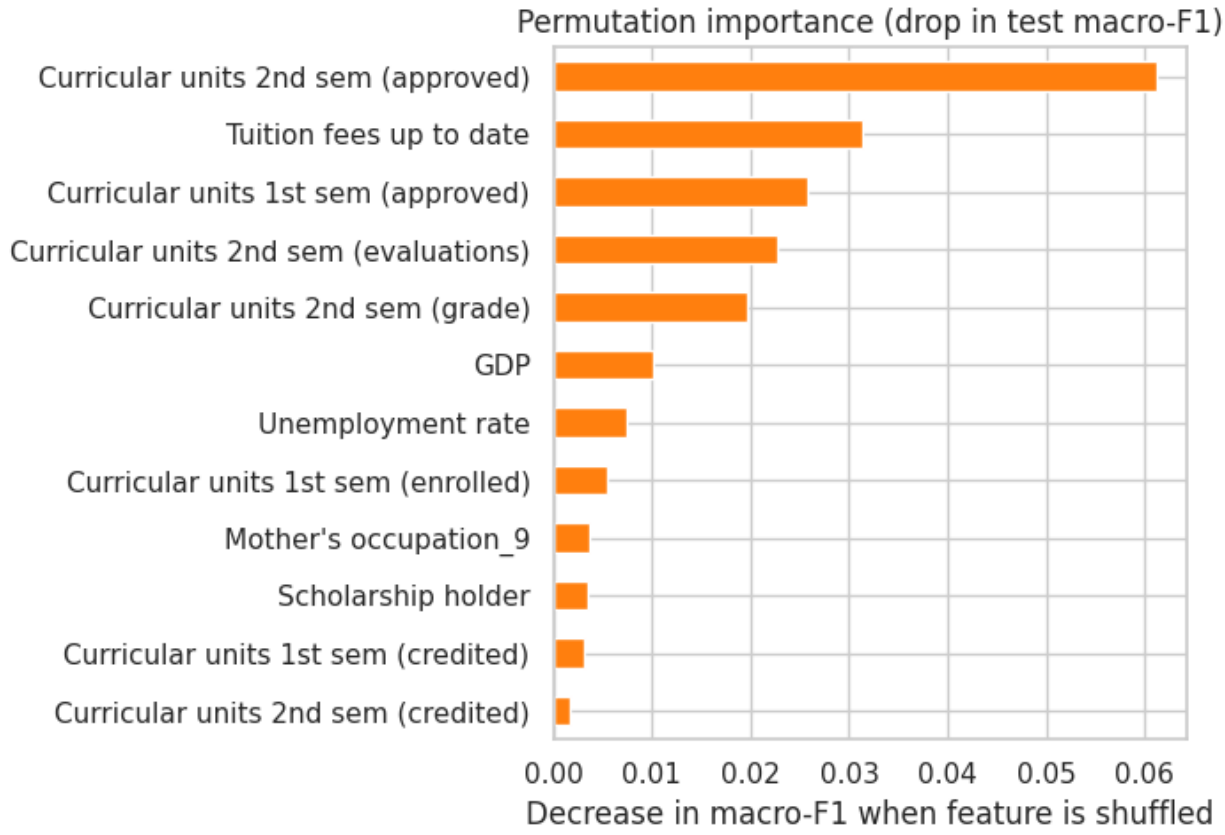


Figure 18 Permutation importance on the test set (drop in macro-F1 when a feature is shuffled).

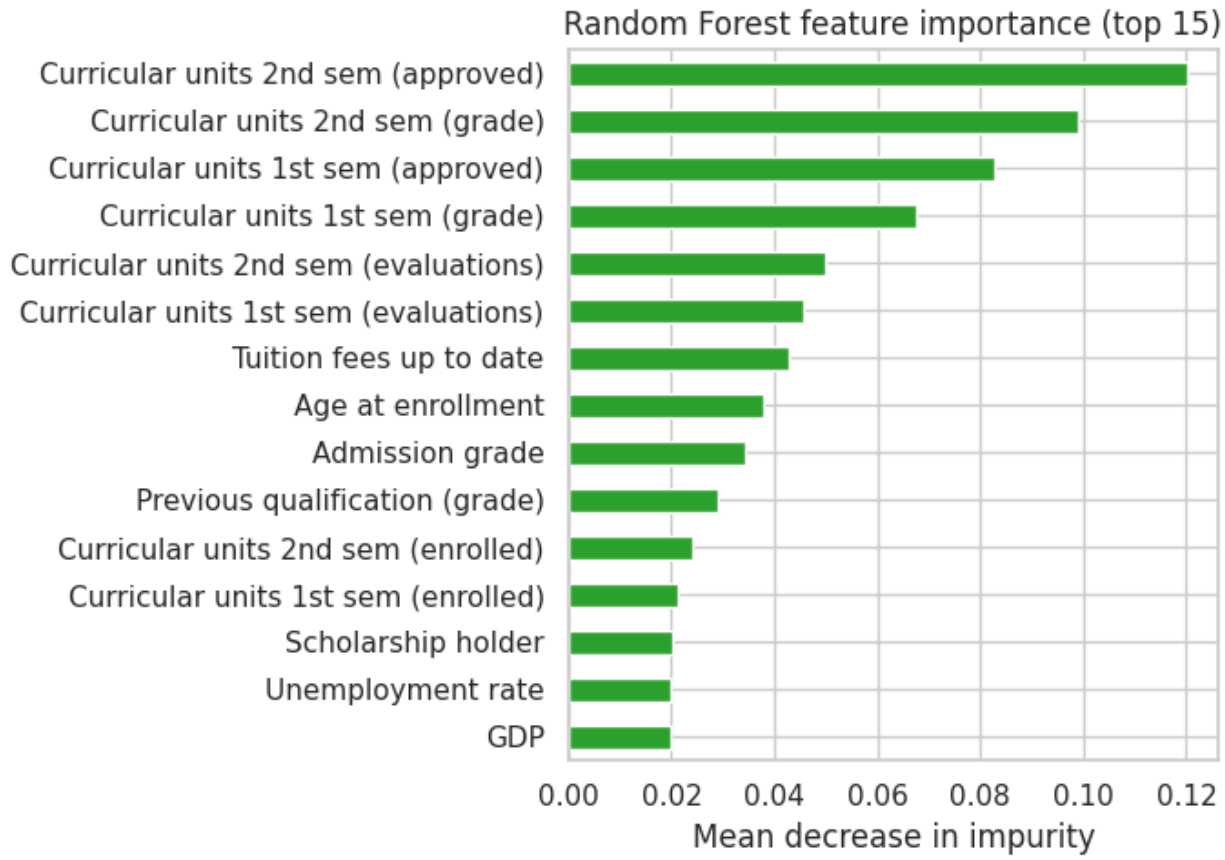


Figure 19 Permutation importance on the test set (drop in macro-F1 when a feature is shuffled)

Both plots show that academic performance drives the model. In the impurity plot the top four features are 2nd-semester approved units (0.120), 2nd-semester grade (0.099), 1st-semester approved units (0.083) and 1st-semester grade (0.068). Permutation importance agrees on the top feature: shuffling 2nd-semester approved units causes by far the largest drop in macro-F1 (about 0.06). Tuition fees up to date ranks second under permutation (about 0.03) but only seventh by impurity, so it matters more for actual predictions than its split counts suggest. Demographic features contribute little. Permutation importance is the more reliable guide, because impurity importance can favour features with many distinct values.

7. Comparison of the two algorithms

7.1 Design differences

Aspect	Logistic Regression	Random Forest
Model type	Single linear model (softmax over class scores)	Ensemble of many decision trees (bagging)
Relationship captured	Linear in the log-odds	Non-linear, including feature interactions
Feature scaling	Required (continuous features standardised)	Not required

Categorical codes	One-hot encoding is essential	One-hot preferred, but trees can split on raw codes
Sensitivity to outliers	Moderate (limited by regularisation)	Low (splits depend on ordering, not scale)
Control of overfitting	Regularisation strength C (L1/L2)	Tree depth, minimum leaf size, number of trees
Main hyperparameters	C, class_weight	n_estimators, max_depth, min_samples_leaf, max_features, class_weight
Interpretability	High: signed coefficients per feature	Lower: feature importance only (no direction)
Training / prediction cost	Low	Higher (many trees)

Table 6 How the two algorithms differ in design and use.

7.2 Test-set accuracy and related metrics

Metric	Logistic Regression (tuned)	Random Forest (tuned)	Difference (RF − LR)
Accuracy	0.734	0.759	+0.025
Macro-F1	0.696	0.703	+0.006
Macro-precision	0.704	0.707	+0.003
Macro-recall	0.708	0.701	−0.007
Macro-AUC (one-vs-rest)	0.884	0.887	+0.002
F1 – Dropout	0.751	0.760	+0.009
F1 – Enrolled	0.504	0.491	−0.013
F1 – Graduate	0.834	0.857	+0.023
Recall – Enrolled	0.635	0.509	−0.126
Cross-validated macro-F1 (training data)	0.715 ± 0.015	0.717	+0.002
Best hyperparameters	C = 0.1, class_weight = "balanced"	n_estimators = 300, max_depth = 16, min_samples_leaf = 2, max_features = "sqrt", class_weight = "balanced"	-

Table 7 Test-set results on the same split (random_state = 42, stratified 80/20). Differences are computed from

7.3 Is the difference meaningful?

With only 885 test students, a single random split can exaggerate or hide differences, so three statistical checks were added (all in the Logistic Regression notebook, using the stored Random Forest settings; the reproduced Random Forest matches the saved results exactly).

Check	Result	Interpretation
McNemar exact test (accuracy)	RF correct & LR wrong = 71; LR correct & RF wrong = 49; p = 0.0548	Borderline: just above 0.05
Paired bootstrap (2000 resamples), accuracy difference RF − LR	mean +0.0247, 95% interval [+0.000, +0.049]	Interval only just touches 0
Paired bootstrap, macro-F1 difference	mean +0.0062, 95% interval [−0.022, +0.035]	Includes 0: a tie
Paired bootstrap, Enrolled-F1 difference	mean −0.0137, 95% interval [−0.070, +0.044]	Includes 0: a tie
Repeated 5-fold CV × 3 on training data	Accuracy 0.754 ± 0.011 (LR) vs 0.770 ± 0.013 (RF), RF ahead in 14 of 15 folds; macro-F1 0.712 ± 0.011 vs 0.714 ± 0.017, RF ahead in 11 of 15 folds	RF is consistently a little more accurate; macro-F1 is a tie

Table 8 Statistical comparison of the two models.

7.4 Discussion of the comparison

• **Overall performance.** Random Forest is slightly ahead in accuracy (0.759 vs 0.734, +0.025) and macro-F1 (0.703 vs 0.696). The macro-F1 gap is far smaller than the cross-validation standard deviation (about 0.015), so on this metric the models tie. The accuracy advantage is suggestive but not conclusive ($p \approx 0.055$), although it is consistent across repeated cross-validation folds.

• The "Enrolled" class is hardest for both models (F1 0.504 vs 0.491, a tie), but they fail differently: Logistic Regression finds more Enrolled students (recall 0.635 vs 0.509) at the cost of more false alarms (precision 0.417 vs 0.474).

• **Dropout and Graduate classes.** Random Forest has the higher F1 for both (0.760 vs 0.751; 0.857 vs 0.834). Most of its accuracy gain comes from Graduate recall (0.878 vs 0.799). AUC values are almost identical (0.887 vs 0.884), so both rank students by risk equally well.

• **Stability.** Cross-validated and test macro-F1 are close for both models (LR 0.715 → 0.696; RF 0.717 → 0.703); neither shows serious overfitting.

• **Interpretability vs. performance.** Logistic Regression gives signed coefficients an institution can read directly. Random Forest gives importance without direction and is slower, but can capture non-linear interactions. Because the performance gap is small, interpretability is a real argument for Logistic Regression.

• **Recommendation**. If the goal is the best overall accuracy, Random Forest is the slightly better choice. If the institution needs to explain why a student is flagged, or wants to find as many "Enrolled" students as possible, Logistic Regression is preferable. Since the macro-F1 difference is a tie and the accuracy difference is small, the choice depends on the use case rather than on accuracy alone.

8. Critical analysis and discussion

8.1 Improvement and ablation experiments

Each member ran a set of experiments (evaluated on the test set for comparison only; no model choice was based on them) to understand what drives performance and how it could be improved.

Experiment	Accuracy	Macro-F 1	Enrolled recall	What it shows
E0 Tuned model (reference)	0.734	0.696	0.635	The final model
E1 Without class_weight	0.776	0.692	0.346	Accuracy rises but only about a third of Enrolled students are found
E2 L1 regularization	0.730	0.692	0.635	Same performance with only 53 of 238 features: a much simpler model
E3 PCA (48 components, 95% variance)	0.729	0.693	0.635	No gain, and interpretability is lost
E4 Early prediction (no semester data)	0.575	0.548	0.497	Performance collapses without academic results
E5 Up to 1st semester only	0.710	0.669	0.585	A realistic compromise: predict after one semester

Table 9 – Logistic Regression experiments

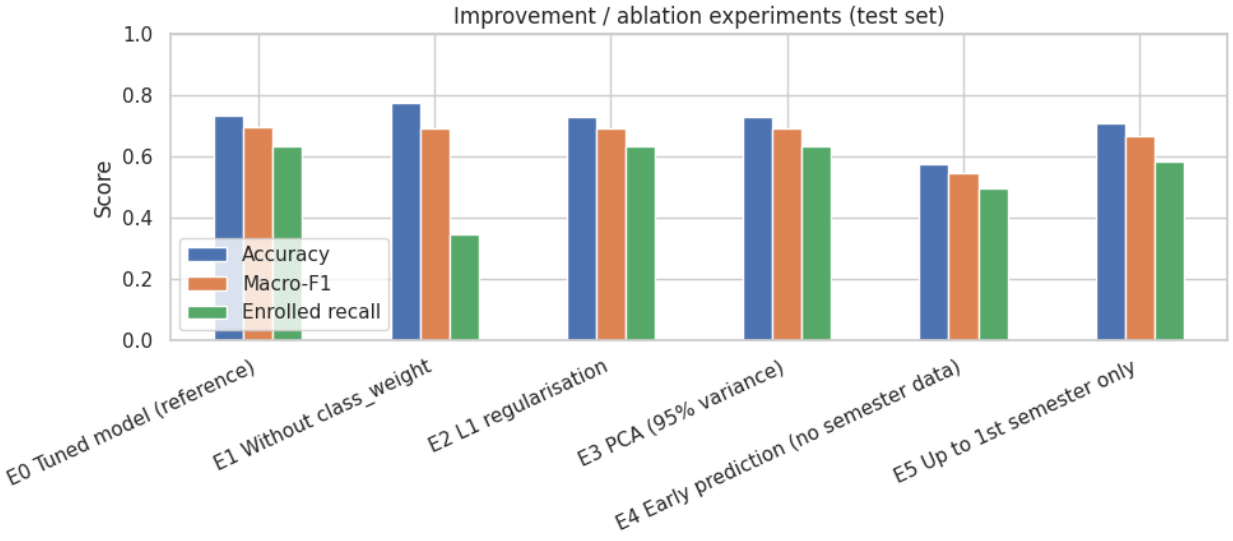


Figure 20 – Logistic Regression improvement / ablation experiments (test set).

Experiment	Accuracy	Macro-F 1	Enrolled recall	What it shows
E0 Tuned model (reference)	0.759	0.703	0.509	The final model
E1 Without class_weight	0.767	0.657	0.226	The model almost ignores the minority class
E2 L1 regularisation	0.766	0.668	0.270	Tuning gains mostly come from class weighting
E3 PCA (48 components)	0.764	0.704	0.484	Same performance with far fewer columns
E4 Early prediction (no semester data)	0.772	0.718	0.522	Best scores; many of the 238 columns add noise (but features were chosen from the data)
E5 Up to 1st semester only	0.615	0.557	0.340	Performance drops sharply
E6 Up to 1st semester only	0.715	0.654	0.428	Recovers much of the performance

Table 10 Random Forest experiments.

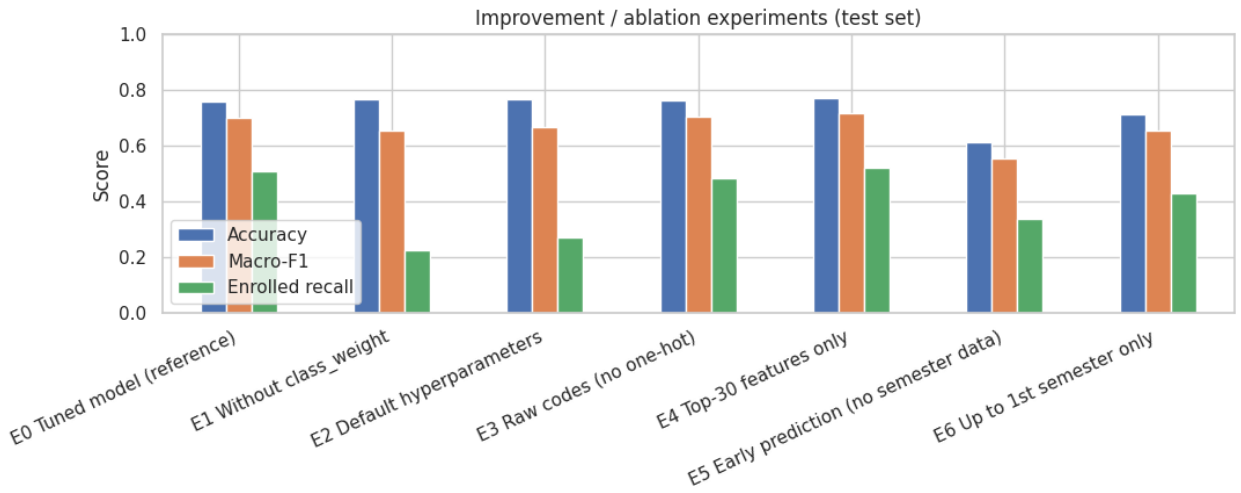


Figure 21 Random Forest improvement / ablation experiments (test set).

8.2 Learning curves

The Logistic Regression validation macro-F1 rises from 0.644 (283 training samples) to about 0.714 at 1,811 samples and then stays flat, while the training score settles near 0.749. The small gap (about 0.035) and the plateau show that more data of the same kind would add little: the limit comes from the model and the features, not the amount of data. For Random Forest the validation macro-F1 is still rising slowly (0.644 to 0.719) and the train/validation gap narrows, so more data would probably help it a little.

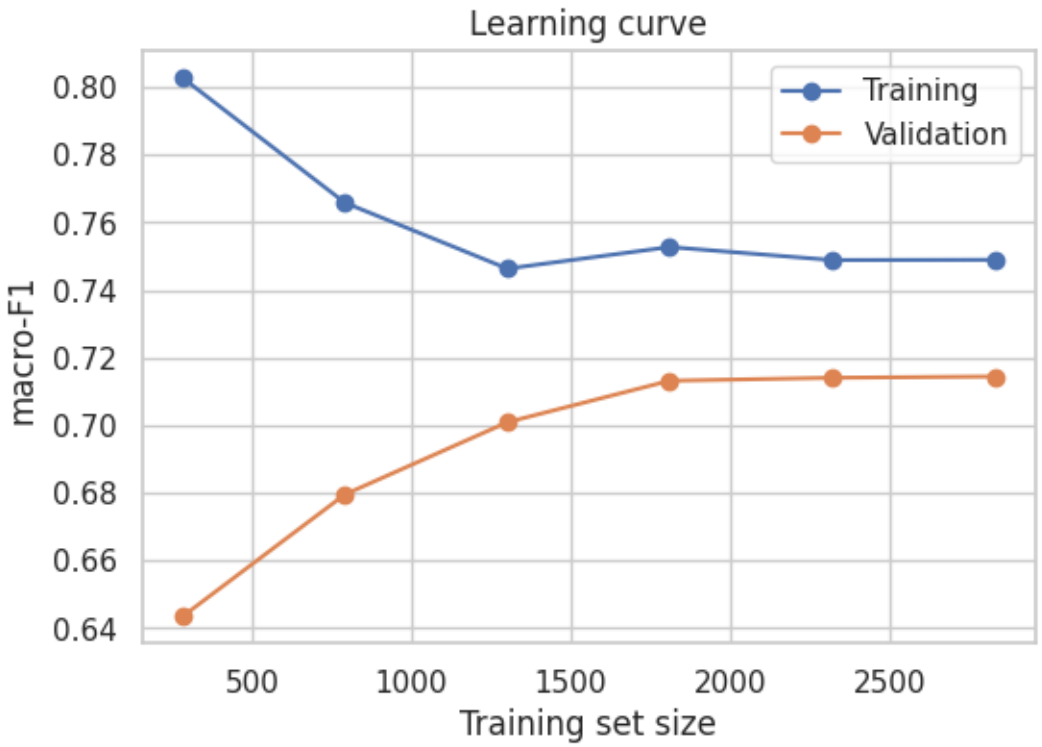


Figure 22 – Logistic Regression learning curve.

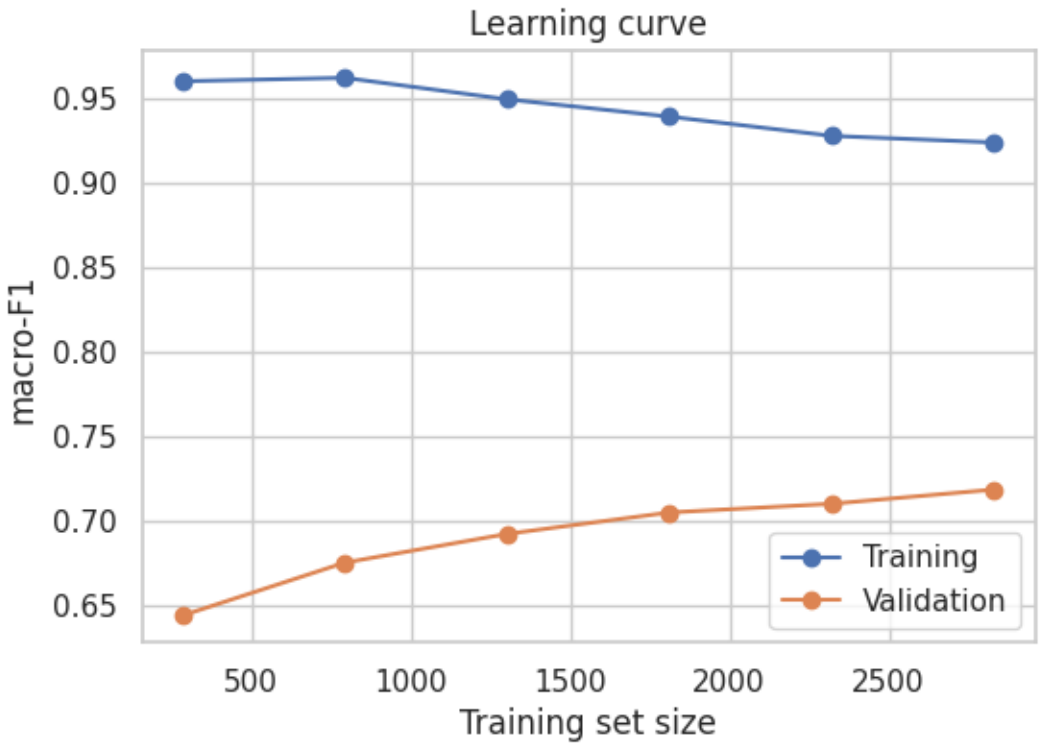


Figure 23 Random Forest learning curve

8.3 Why accuracy is limited

• The “Enrolled” group is intermediate. Its students overlap with both Dropouts and Graduates on almost every feature. For Random Forest, Enrolled students are mostly confused with Graduate (44) and Dropout (34).

• Model assumptions. Logistic Regression is linear and cannot capture interactions such as scholarship × debt × grades. Random Forest can, but the extra flexibility buys little on the balanced metrics.

• Timing of the information. Nearly all the predictive power comes from academic performance after enrolment (E4/E5 for Logistic Regression and E5/E6 for Random Forest). Using only enrolment information, accuracy falls to 57.5% (LR) and 61.5% (RF). A useful early-warning system must work early, so the strong 2nd-semester features are a limitation for practical use.

• Class imbalance. Class weights raise Enrolled recall (LR: 0.35 → 0.64; RF: 0.23 → 0.51) at the cost of overall accuracy, so the best operating point depends on the cost of missing an at-risk student versus a false alarm.

8.4 How the accuracy could be improved

• Engineered features such as approval rate = approved ÷ enrolled units and the grade trend between semesters, and interaction or polynomial features for the linear model.

• Stronger classical models: Gradient Boosting / XGBoost, SVM, or a stacked ensemble of Logistic Regression and Random Forest, which combine linear and non-linear strengths.

• Handling imbalance with resampling (SMOTE) or decision-threshold tuning for the Enrolled class.

• Feature selection: the top-30 feature experiment (E4) gave the best Random Forest scores (accuracy 0.772, macro-F1 0.718), and L1 regularisation kept Logistic Regression performance with 78% fewer features. Because the features were chosen using the data, this must be confirmed inside cross-validation before being adopted.

• More reliable evaluation: repeated or nested cross-validation and a wider hyperparameter search, instead of a single random split.

9. Limitations and future work

9.1 Limitations

• Data come from a single institution and period, so results may not generalise to other universities or countries.

• Class imbalance: “Enrolled” is only 18% of the data and is the hardest to predict.

• Strong dependence on 2nd-semester features, which are available only late in the year.

• Many one-hot columns are rare categories, so some Logistic Regression coefficients are noisy; Random Forest impurity importances are biased towards features with many distinct values and share importance among correlated features.

• The final comparison uses one random train/test split (repeated cross-validation on the training data was added as a second check).

• Logistic Regression assumes a linear relationship with the log-odds; Random Forest is harder to explain to non-technical staff.

9.2 Future work

• Compare with other non-linear classical models (Gradient Boosting, XGBoost, SVM) and a stacked ensemble.

• Build a true early-warning model evaluated at several points in time (enrolment, end of semester 1, end of semester 2).

• Try SMOTE or cost-sensitive learning and decision-threshold tuning for the Enrolled class.

• Engineer more features and test interactions; add SHAP values for per-student explanations.

• Use nested cross-validation for tuning and evaluation in a single unbiased procedure.

• Check fairness across gender, nationality and socio-economic groups before any real-world use.

10. Individual contributions

	Member 1	Member 2
Name	Thuiyadura L.R	Nissanka H.M.D.P
Student ID	MS26923956	MS26930190
Algorithm	Logistic Regression	Random Forest
Notebook	logistic_regression.ipynb	random_forest.ipynb
GitHub branch	thuiyadura-lr-ms26923956	nissanka-hmdp-ms26930190

Shared work (both members)

• Dataset selection and checking it against the assignment requirements.

• Agreeing the shared settings: random_state = 42, stratified 80/20 split, same one-hot encoding, macro-F1 for tuning.

• Repository structure and branches (main, final-development, one branch per member).

• Comparison of the two algorithms (Sections 7 and 8) and the final report, members.txt and submission.txt.

Individual work

Task	Member 1: Logistic Regression	Member 2: Random Forest
Cleansing and outliers	Missing values, duplicates, IQR outlier analysis, decision to keep outliers (limited by L2 regularisation)	Missing values, duplicates, IQR outlier analysis, decision to keep outliers (trees are robust to outliers)
Exploratory analysis	Class distribution, boxplots by outcome, correlation matrix	Class distribution, boxplots by outcome, correlation matrix

Model-specific preprocessing	Standardisation inside a Pipeline (no leakage), PCA analysis	No scaling needed; feature selection by importance instead of PCA
Algorithm background	Logistic Regression (softmax, regularised loss, C)	Random Forest (Gini impurity, bagging, random feature subsets, OOB estimate)
Hyperparameter tuning	GridSearchCV over C and class_weight, scored by macro-F1	RandomizedSearchCV (20 settings, 5-fold CV) over five hyperparameters, scored by macro-F1
Evaluation	Accuracy, per-class precision/recall/F1, macro-F1, confusion matrix, ROC-AUC, cross-validation	Accuracy, per-class precision/recall/F1, macro-F1, confusion matrix, ROC-AUC, cross-validation, OOB accuracy
Interpretation	Coefficients (risk and protective factors)	Impurity-based and permutation feature importance
Experiments	Class weights, L1, PCA, early prediction, 1st-semester only, learning curve	Class weights, default hyperparameters, raw codes vs one-hot, top-30 features, early prediction, 1st-semester only, number of trees, learning curve
Statistical comparison	McNemar test, paired bootstrap and repeated CV	Tuned Random Forest and its results file, plus the comparison table in its notebook
Results file	results/logistic_regression_results.json	results/random_forest_results.json

11. Conclusion

Two distinct supervised algorithms were applied to the UCI student dropout dataset under an identical protocol. Tuned Logistic Regression predicts student outcomes with about 73% accuracy, macro-F1 0.70 and macro-AUC 0.884; tuned Random Forest reaches 76% accuracy, macro-F1 0.70 and macro-AUC 0.887. Both identify Graduates and Dropouts well and struggle with “Enrolled” students, and almost all of their predictive power comes from academic results achieved after enrolment. Random Forest is slightly more accurate (+2.5 points; borderline significance), but on macro-F1 and AUC the two are effectively tied, so the extra complexity buys little on the balanced metrics. Logistic Regression’s main strengths are interpretability – its coefficients point to tuition-payment status and approved curricular units as the factors most associated with dropout and higher recall on the hard Enrolled class. Improvement experiments show that the limit lies in the available features and the timing of the data rather than in the amount of data or the choice between these two models, so engineered features, boosted ensembles and genuine early-warning evaluation are the most promising next steps.

12. References

[1] Realinho, V., Vieira Martins, M., Machado, J., & Baptista, L. (2021). Predict Students' Dropout and Academic Success [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89 [2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Appendix A – Source code: Logistic Regression notebook (logistic_regression.ipynb)

APPENDIX A - Source code: Logistic Regression notebook (logistic_regression.ipynb)

Paste this as TEXT (monospace font, e.g. Consolas 8 pt) into Appendix A of the report. Do not paste screenshots.

==========

```
# ---------------- Cell 1 ----------------

# Clone the project repository (it contains the dataset in data/data.csv)

!git clone -b thuiyadura-lr-ms26923956 https://github.com/LoshithaAI/ML-Assignment.git


# ---------------- Cell 2 ----------------

# List the repository contents to confirm the clone worked

!ls ML-Assignment


# ---------------- Cell 3 ----------------

# --- Libraries ----------------------------------------------------------

import warnings

import json

import numpy as np              # numerical arrays

import pandas as pd             # tables (DataFrames)

import matplotlib.pyplot as plt     # plotting

import seaborn as sns           # statistical plots (boxplots, heatmap)


# scikit-learn building blocks, grouped by purpose

from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold, cross_val_score, learning_curve  # splitting, tuning, validation
```

```
from sklearn.preprocessing import StandardScaler, label_binarize   # scaling; one-vs-rest labels for the ROC curves
from sklearn.compose import ColumnTransformer                      # apply scaling to selected columns only
from sklearn.pipeline import Pipeline                              # chain preprocessing + model (prevents data leakage)
from sklearn.decomposition import PCA                              # dimensionality-reduction analysis
from sklearn.linear_model import LogisticRegression               # the algorithm of this notebook
from sklearn.metrics import (accuracy_score, f1_score, precision_score, recall_score,      # evaluation metrics
                             classification_report, confusion_matrix, ConfusionMatrixDisplay,
                             roc_auc_score, roc_curve)

warnings.filterwarnings("ignore")   # hide library deprecation warnings so the output stays readable
sns.set_theme(style="whitegrid")    # one consistent plot style for all figures
RANDOM_STATE = 42                   # fixed seed: results are reproducible and identical for both team members


# ---------------- Cell 4 ----------------
# --- Load the dataset -------------------------------------------------------
import os
GITHUB_RAW_URL = "https://raw.githubusercontent.com/LoshithaAI/ML-Assignment/thuiyadura-lr-ms26923956/data/data.csv"

# Look for a local copy first: it exists after the clone above, or when the notebook is run from the repository.
# The "else" of a for-loop runs only if the loop finished without "break", i.e. no local file was found.
for path in ["data.csv", "data/data.csv", "../data/data.csv", "ML-Assignment/data/data.csv"]:
    if os.path.exists(path):
        DATA_PATH = path
        break
else:
```

```
    DATA_PATH = GITHUB_RAW_URL


# The UCI file is separated by semicolons, so sep=";" is required (the default comma would give one very wide column).
df = pd.read_csv(DATA_PATH, sep=";")
df.columns = df.columns.str.strip()
print("Loaded from:", DATA_PATH)
print("Shape:", df.shape)


# ---------- Cell 5 ----------
# Record library versions for reproducibility
import sklearn, sys
print("Python      :", sys.version.split()[0])
print("pandas      :", pd.__version__)
print("numpy       :", np.__version__)
print("scikit-learn:", sklearn.__version__)


# ---------- Cell 6 ----------
# First rows of the dataset (all columns are numeric except Target)
display(df.head())
print("Rows, columns:", df.shape)
print("Features:", df.shape[1] - 1, "| Target: Target")


# ---------- Cell 7 ----------
# Class distribution of the target
counts = df["Target"].value_counts()
print(pd.DataFrame({"count": counts, "percent": (counts / len(df) * 100).round(1)}))


plt.figure(figsize=(5, 3.5))
```

```
sns.countplot(data=df, x="Target", order=counts.index)
plt.title("Class distribution of the target")
plt.show()

# ---------- Cell 8 ----------
# Summary statistics of the continuous attributes
cat_cols = ["Marital status", "Application mode", "Course", "Previous qualification", "Nacionality",
        "Mother's qualification", "Father's qualification", "Mother's occupation", "Father's occupation"]
binary_cols = [c for c in df.columns if c not in cat_cols + ["Target"] and df[c].nunique() <= 2]
num_cols = [c for c in df.columns if c not in cat_cols + binary_cols + ["Target", "Application order"]]

print("Categorical-code columns :", len(cat_cols))
print("Binary (0/1) columns     :", len(binary_cols))
print("Continuous columns       :", len(num_cols))
# summary statistics (mean, spread, min/max) of the continuous columns
df[num_cols].describe().T.round(2)

# ---------- Cell 9 ----------
#Check for missing values, duplicate rows and non-numeric columns
print("Missing values (total):", int(df.isna().sum().sum()))
print("Duplicate rows       :", int(df.duplicated().sum()))
print("Non-numeric columns   :", list(df.select_dtypes(exclude="number").columns))

# ---------- Cell 10 ----------
# IQR rule: values outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR] are flagged as potential outliers
rows = []
for c in num_cols:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
```

```
    n_out = int(((df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)).sum())
    rows.append((c, n_out, round(n_out / len(df) * 100, 1)))
outliers = pd.DataFrame(rows, columns=["feature", "n_outliers", "percent"]).sort_values("n_outliers",
ascending=False)
display(outliers.head(8))


fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
for ax, c in zip(axes, ["Age at enrollment", "Admission grade", "Curricular units 1st sem (grade)"]):
    sns.boxplot(y=df[c], ax=ax)
    ax.set_title(c)
plt.tight_layout()
plt.show()


# ---------- Cell 11 ----------
# One boxplot per feature, split by Target. Clearly different medians suggest a useful predictor.
fig, axes = plt.subplots(1, 3, figsize=(14, 3.8))
for ax, c in zip(axes, ["Curricular units 1st sem (approved)",
                        "Curricular units 2nd sem (approved)",
                        "Admission grade"]):
    sns.boxplot(data=df, x="Target", y=c, order=["Dropout", "Enrolled", "Graduate"], ax=ax)
    ax.set_title(c, fontsize=10)
plt.tight_layout()
plt.show()


# ---------- Cell 12 ----------
# Correlation between the continuous attributes
plt.figure(figsize=(11, 8))
sns.heatmap(df[num_cols].corr(), cmap="coolwarm", center=0, annot=False)
plt.title("Correlation matrix of continuous features")
```

```
plt.show()


# ---------- Cell 13 ----------
# One-hot encode the 9 code-valued columns; the other 27 features stay unchanged
X = pd.get_dummies(df.drop(columns="Target"), columns=cat_cols, drop_first=True)
y = df["Target"]
print("Features before encoding:", df.shape[1] - 1)
print("Features after encoding :", X.shape[1])


# ---------- Cell 14 ----------
# 80/20 split, STRATIFIED so each part keeps the same class proportions.
# Use exactly the same settings as your teammate so the models are compared fairly.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)


print("Train:", X_train.shape, "| Test:", X_test.shape)
print("Train class %:", (y_train.value_counts(normalize=True) * 100).round(1).to_dict())
print("Test  class %:", (y_test.value_counts(normalize=True) * 100).round(1).to_dict())


# ---------- Cell 15 ----------
def build_pipeline(scale_cols, C=1.0, class_weight=None, penalty="l2", solver="lbfgs", max_iter=5000):
    """Scaling (on continuous columns only) + Logistic Regression in one pipeline."""
    scaler = ColumnTransformer([("scale", StandardScaler(), scale_cols)], remainder="passthrough")
    clf = LogisticRegression(C=C, class_weight=class_weight, penalty=penalty,
                             solver=solver, max_iter=max_iter, random_state=RANDOM_STATE)
    return Pipeline([("prep", scaler), ("clf", clf)])


print("Continuous columns that will be scaled:", len(num_cols))
```

```
# ---------------- Cell 16 ----------------
# How many principal components are needed to keep 95% of the variance?
prep_only = ColumnTransformer([("scale", StandardScaler(), num_cols)], remainder="passthrough")
Xtr_scaled = prep_only.fit_transform(X_train)
pca_full = PCA().fit(Xtr_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
n95 = int(np.argmax(cum_var >= 0.95) + 1)
print(f"{X_train.shape[1]} features -> {n95} components keep 95% of the variance")

plt.figure(figsize=(6, 3.8))
plt.plot(range(1, len(cum_var) + 1), cum_var)
plt.axhline(0.95, color="red", ls="--", label="95%")
plt.axvline(n95, color="gray", ls="--")
plt.xlabel("Number of components"); plt.ylabel("Cumulative explained variance")
plt.title("PCA on the encoded training data"); plt.legend()
plt.show()

# ---------------- Cell 17 ----------------
# Baseline model: default settings (C=1, no class weights) as a reference point
baseline = build_pipeline(num_cols)
baseline.fit(X_train, y_train)
pred_base = baseline.predict(X_test)

base_acc = accuracy_score(y_test, pred_base)
base_f1 = f1_score(y_test, pred_base, average="macro")
print(f"Baseline  accuracy = {base_acc:.3f} | macro-F1 = {base_f1:.3f}")

# ---------------- Cell 18 ----------------
# Grid search over C and class_weight: 5-fold stratified CV on the training set, selected by macro-F1
```

```
param_grid = {"clf__C": [0.001, 0.01, 0.1, 1, 10, 100],
              "clf__class_weight": [None, "balanced"]}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
grid = GridSearchCV(build_pipeline(num_cols), param_grid, cv=cv,
                    scoring="f1_macro", n_jobs=-1, return_train_score=True)
grid.fit(X_train, y_train)

print("Best parameters      :", grid.best_params_)
print(f"Best CV macro-F1     : {grid.best_score_:.3f}")

cv_results = (pd.DataFrame(grid.cv_results_)
              [["param_clf__C", "param_clf__class_weight", "mean_train_score", "mean_test_score",
"std_test_score"]]
              .sort_values("mean_test_score", ascending=False))
display(cv_results.round(3).head(8))

# ---------- Cell 19 ----------
# Effect of C on cross-validated macro-F1 (shows under- and over-fitting)
plt.figure(figsize=(6.5, 4))
for cw in [None, "balanced"]:
    sub = cv_results[cv_results["param_clf__class_weight"].astype(str) ==
str(cw)].sort_values("param_clf__C")
    plt.semilogx(sub["param_clf__C"].astype(float), sub["mean_test_score"], marker="o",
label=f"validation (class_weight={cw})")
    plt.semilogx(sub["param_clf__C"].astype(float), sub["mean_train_score"], ls="--", alpha=0.5,
label=f"train (class_weight={cw})")
plt.xlabel("C (log scale)"); plt.ylabel("macro-F1"); plt.legend(fontsize=8)
plt.title("Hyperparameter C vs. macro-F1")
plt.show()
```

```
# ---------------- Cell 20 ----------------
# Final tuned model: predict on the held-out test set (used once)
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)
classes = list(best_model.classes_)

test_acc = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred, average="macro")
print(f"Test accuracy : {test_acc:.3f}")
print(f"Test macro-F1 : {test_f1:.3f}")
print()
print(classification_report(y_test, y_pred, digits=3))

# ---------------- Cell 21 ----------------
# Confusion matrices: raw counts and row-normalised (recall per class)
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axes[0], cmap="Blues", colorbar=False)
axes[0].set_title("Confusion matrix (counts)")
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, normalize="true", ax=axes[1],
                                        cmap="Blues", colorbar=False, values_format=".2f")
axes[1].set_title("Confusion matrix (row-normalised)")
plt.tight_layout()
plt.show()

# ---------------- Cell 22 ----------------
# One-vs-rest ROC curves and AUC
y_bin = label_binarize(y_test, classes=classes)
```

```
plt.figure(figsize=(6, 4.5))
for i, c in enumerate(classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
    plt.plot(fpr, tpr, label=f"{c} (AUC = {roc_auc_score(y_bin[:, i], y_proba[:, i]):.3f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.4)
plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
plt.title("ROC curves (one-vs-rest)"); plt.legend()
plt.show()

macro_auc = roc_auc_score(y_test, y_proba, multi_class="ovr", average="macro")
print(f"Macro-average AUC (OvR): {macro_auc:.3f}")

# ---------- Cell 23 ----------
# Stability check: 5-fold CV of the final model on the TRAINING data
cv_acc = cross_val_score(best_model, X_train, y_train, cv=cv, scoring="accuracy")
cv_f1 = cross_val_score(best_model, X_train, y_train, cv=cv, scoring="f1_macro")
print(f"CV accuracy : {cv_acc.mean():.3f} +/- {cv_acc.std():.3f}")
print(f"CV macro-F1 : {cv_f1.mean():.3f} +/- {cv_f1.std():.3f}")

# ---------- Cell 24 ----------
# Summary table: baseline vs. tuned model
summary = pd.DataFrame({
    "Model": ["Logistic Regression (baseline)", "Logistic Regression (tuned)"],
    "Accuracy": [base_acc, test_acc],
    "Macro-F1": [base_f1, test_f1],
    "Macro-Precision": [precision_score(y_test, pred_base, average="macro"), precision_score(y_test, y_pred, average="macro")],
    "Macro-Recall": [recall_score(y_test, pred_base, average="macro"), recall_score(y_test, y_pred, average="macro")],
```

```
}).round(3)
summary

# ---------- Cell 25 ----------
# Coefficients per class: positive raises that class's score, negative lowers it
feature_names = [n.split("__", 1)[1] for n in best_model.named_steps["prep"].get_feature_names_out()]
coefs = pd.DataFrame(best_model.named_steps["clf"].coef_.T, index=feature_names, columns=classes)

top = coefs["Dropout"].sort_values()
print("Features that most REDUCE the odds of Dropout:")
display(top.head(8).round(3))
print("Features that most INCREASE the odds of Dropout:")
display(top.tail(8).round(3))

plt.figure(figsize=(7, 5))
pd.concat([top.head(8), top.tail(8)]).plot(kind="barh", color=["tab:green"] * 8 + ["tab:red"] * 8)
plt.title("Largest Logistic Regression coefficients (Dropout class)")
plt.xlabel("Coefficient (on standardised scale for continuous features)")
plt.tight_layout()
plt.show()

# ---------- Cell 26 ----------
# Save the test metrics to a JSON file so they can be compared with the Random Forest results
RESULTS_DIR = "/content/ML-Assignment/results"
os.makedirs(RESULTS_DIR, exist_ok=True)

RESULTS_FILE = os.path.join(
    RESULTS_DIR, "logistic_regression_results.json"
)
```

```
results = {
    "model": "Logistic Regression",
    "best_params": {k.replace("clf__", ""): (None if v is None else (float(v) if isinstance(v, (int, float, np.floating)) else v))
                    for k, v in grid.best_params_.items()},
    "test_accuracy": round(float(test_acc), 4),
    "test_macro_f1": round(float(test_f1), 4),
    "test_macro_auc_ovr": round(float(macro_auc), 4),
    "cv_macro_f1_mean": round(float(cv_f1.mean()), 4),
    "per_class": classification_report(y_test, y_pred, output_dict=True, digits=4),
}
with open(RESULTS_FILE, "w") as f:
    json.dump(results, f, indent=2)
print("Saved to:", RESULTS_FILE)


# ---------- Cell 27 ----------
#download the results file
try:
    from google.colab import files
    files.download(RESULTS_FILE)
except ImportError:
    pass


# ---------- Cell 28 ----------
# Improvement experiments (evaluated on the test set for comparison only)
best_C = grid.best_params_["clf__C"]
best_cw = grid.best_params_["clf__class_weight"]
experiments = {}
```

```
def evaluate(name, model, Xtr, Xte):
    model.fit(Xtr, y_train)
    p = model.predict(Xte)
    experiments[name] = {"Accuracy": accuracy_score(y_test, p),
                         "Macro-F1": f1_score(y_test, p, average="macro"),
                         "Enrolled recall": recall_score(y_test, p, labels=["Enrolled"], average=None)[0]}

evaluate("E0 Tuned model (reference)", build_pipeline(num_cols, C=best_C, class_weight=best_cw),
X_train, X_test)

evaluate("E1 Without class_weight", build_pipeline(num_cols, C=best_C, class_weight=None), X_train,
X_test)

l1 = build_pipeline(num_cols, C=best_C, class_weight=best_cw, penalty="l1", solver="saga",
max_iter=3000)
evaluate("E2 L1 regularisation", l1, X_train, X_test)
n_kept = int((np.abs(l1.named_steps["clf"].coef_).max(axis=0) > 1e-8).sum())
print(f"L1 kept {n_kept} of {X_train.shape[1]} features (the rest have zero weight)")

pca_pipe = Pipeline([("prep", ColumnTransformer([("scale", StandardScaler(), num_cols)],
remainder="passthrough")),
                     ("pca", PCA(n_components=0.95, random_state=RANDOM_STATE)),
                     ("clf", LogisticRegression(C=best_C, class_weight=best_cw, max_iter=5000,
random_state=RANDOM_STATE))])
evaluate("E3 PCA (95% variance)", pca_pipe, X_train, X_test)
print("PCA components used:", pca_pipe.named_steps["pca"].n_components_)

early_cols = [c for c in X_train.columns if not c.startswith("Curricular units")]
early_num = [c for c in num_cols if c in early_cols]
evaluate("E4 Early prediction (no semester data)", build_pipeline(early_num, C=best_C,
class_weight=best_cw),
```

```
        X_train[early_cols], X_test[early_cols])

sem1_cols = [c for c in X_train.columns if not c.startswith("Curricular units 2nd sem")]
sem1_num = [c for c in num_cols if c in sem1_cols]
evaluate("E5 Up to 1st semester only", build_pipeline(sem1_num, C=best_C, class_weight=best_cw),
        X_train[sem1_cols], X_test[sem1_cols])

exp_table = pd.DataFrame(experiments).T.round(3)
exp_table

# ---------- Cell 29 ----------
# Bar chart comparing the experiments
exp_table[["Accuracy", "Macro-F1", "Enrolled recall"]].plot(kind="bar", figsize=(10, 4.5))
plt.title("Improvement / ablation experiments (test set)")
plt.ylabel("Score"); plt.ylim(0, 1); plt.xticks(rotation=25, ha="right")
plt.legend(loc="lower left")
plt.tight_layout()
plt.show()

# ---------- Cell 30 ----------
# Learning curve: does more training data help
sizes, tr_scores, va_scores = learning_curve(
    build_pipeline(num_cols, C=best_C, class_weight=best_cw), X_train, y_train,
    train_sizes=np.linspace(0.1, 1.0, 6), cv=cv, scoring="f1_macro", n_jobs=-1)

plt.figure(figsize=(6, 4))
plt.plot(sizes, tr_scores.mean(axis=1), marker="o", label="Training")
plt.plot(sizes, va_scores.mean(axis=1), marker="o", label="Validation")
plt.xlabel("Training set size"); plt.ylabel("macro-F1"); plt.legend()
```

```
plt.title("Learning curve")
plt.show()


# ---------- Cell 31 ----------
import json, os

RF_RESULTS_PATH = "/content/ML-Assignment/results/random_forest_results.json"
LR_RESULTS_PATHS = ["results/logistic_regression_results.json",
                    "/content/results/logistic_regression_results.json",
                    "/content/ML-Assignment/results/logistic_regression_results.json"]

def load_json(paths):
    """Return the contents of the first results file that exists, or None."""
    for path in ([paths] if isinstance(paths, str) else paths):
        if os.path.exists(path):
            with open(path) as f:
                return json.load(f)
    return None

def summarise(r):
    """Pick the metrics used in Table B from one results file."""
    if r is None:
        return {k: "TBD" for k in ["Accuracy", "Macro-F1", "Macro-precision", "Macro-recall", "Macro-AUC
(one-vs-rest)",
                                   "F1 - Dropout", "F1 - Enrolled", "F1 - Graduate", "Recall - Enrolled",
                                   "Cross-validated macro-F1 (training data)", "Best hyperparameters"]}
    pc = r["per_class"]
    return {"Accuracy": round(pc["accuracy"], 3),
            "Macro-F1": round(pc["macro avg"]["f1-score"], 3),
```

```
        "Macro-precision": round(pc["macro avg"]["precision"], 3),
        "Macro-recall": round(pc["macro avg"]["recall"], 3),
        "Macro-AUC (one-vs-rest)": round(r["test_macro_auc_ovr"], 3),
        "F1 - Dropout": round(pc["Dropout"]["f1-score"], 3),
        "F1 - Enrolled": round(pc["Enrolled"]["f1-score"], 3),
        "F1 - Graduate": round(pc["Graduate"]["f1-score"], 3),
        "Recall - Enrolled": round(pc["Enrolled"]["recall"], 3),
        "Cross-validated macro-F1 (training data)": round(r["cv_macro_f1_mean"], 3),
        "Best hyperparameters": str(r["best_params"])}

lr_res, rf_res = load_json(LR_RESULTS_PATHS), load_json(RF_RESULTS_PATH)
table_b = pd.DataFrame({"Logistic Regression (tuned)": summarise(lr_res),
                        "Random Forest (tuned)": summarise(rf_res)})
display(table_b)

if lr_res is None:
    print("logistic_regression_results.json not found: run the cell that saves the results first.")
if rf_res is None:
    print(f"Random Forest results not found at {RF_RESULTS_PATH}. Upload random_forest_results.json to
that folder "
          "(Files panel -> ML-Assignment -> results) and run this cell again.")
if lr_res is not None and rf_res is not None:
    # Draft facts for the discussion points (edit them into your own words)
    d_acc = rf_res["per_class"]["accuracy"] - lr_res["per_class"]["accuracy"]
    d_f1 = rf_res["per_class"]["macro avg"]["f1-score"] - lr_res["per_class"]["macro avg"]["f1-score"]
    better_acc = "Random Forest" if d_acc > 0 else "Logistic Regression"
    better_f1 = "Random Forest" if d_f1 > 0 else "Logistic Regression"
    print(f"1. Accuracy : {better_acc} is higher by {abs(d_acc):.3f}")
```

```
    print(f"   Macro-F1 : {better_f1} is higher by {abs(d_f1):.3f}" + ("  -> within 0.015, treat as a tie" if abs(d_f1) < 0.015 else ""))
    for cls in ["Enrolled", "Dropout", "Graduate"]:
        a, b = lr_res["per_class"][cls]["f1-score"], rf_res["per_class"][cls]["f1-score"]
        print(f"   F1 {cls:8s}: LR {a:.3f} | RF {b:.3f}")
    print(f"4. CV vs test macro-F1: LR {lr_res['cv_macro_f1_mean']:.3f} -> {lr_res['test_macro_f1']:.3f} | "
          f"RF {rf_res['cv_macro_f1_mean']:.3f} -> {rf_res['test_macro_f1']:.3f}")


# ---------- Cell 32 ----------
# Statistical check: is the difference between the two models meaningful
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import binomtest

if rf_res is None:
    print("random_forest_results.json is needed: run the previous cell with the file available first.")
else:
    rf_model = RandomForestClassifier(**rf_res["best_params"], random_state=RANDOM_STATE, n_jobs=-1).fit(X_train, y_train)
    pred_rf, pred_lr = rf_model.predict(X_test), y_pred
    print(f"Reproduced Random Forest: accuracy {accuracy_score(y_test, pred_rf):.4f} (file {rf_res['test_accuracy']}), "
          f"macro-F1 {f1_score(y_test, pred_rf, average='macro'):.4f} (file {rf_res['test_macro_f1']})")

    yt = y_test.values
    ok_rf, ok_lr = (pred_rf == yt), (pred_lr == yt)
    only_rf, only_lr = int((ok_rf & ~ok_lr).sum()), int((~ok_rf & ok_lr).sum())
    print(f"\nMcNemar test: RF correct & LR wrong = {only_rf}, LR correct & RF wrong = {only_lr}")
    print(f"exact p-value = {binomtest(only_rf, only_rf + only_lr, 0.5).pvalue:.4f}  (p < 0.05 would mean a significant difference in accuracy)")
```

```
    # Paired bootstrap: resample the test set 2000 times and recompute the RF - LR difference each time
    rng, n = np.random.RandomState(0), len(yt)
    diffs = {"accuracy": [], "macro-F1": [], "Enrolled F1": []}
    for _ in range(2000):
        idx = rng.randint(0, n, n)
        diffs["accuracy"].append(accuracy_score(yt[idx], pred_rf[idx]) - accuracy_score(yt[idx], pred_lr[idx]))
        diffs["macro-F1"].append(f1_score(yt[idx], pred_rf[idx], average="macro") - f1_score(yt[idx],
pred_lr[idx], average="macro"))
        diffs["Enrolled F1"].append(f1_score(yt[idx], pred_rf[idx], labels=["Enrolled"], average=None)[0]
                                    - f1_score(yt[idx], pred_lr[idx], labels=["Enrolled"], average=None)[0])
    print("\nPaired bootstrap, RF minus LR (95% interval):")
    for k, d in diffs.items():
        lo, hi = np.percentile(d, [2.5, 97.5])
        print(f"  {k:12s}: mean {np.mean(d):+.4f}   [{lo:+.4f}, {hi:+.4f}]   " + ("excludes 0" if lo > 0 or hi < 0 else
"includes 0 -> not a clear difference"))
```

Appendix B – Source code: Random Forest notebook (random_forest.ipynb)

```
# ---------------- Cell 01 ----------------
# Clone the project repository (it contains the dataset in data/data.csv)
!git clone -b nissanka-hmdp-ms26930190 https://github.com/LoshithaAI/ML-Assignment.git

# ---------------- Cell 02 ----------------
# List the repository contents to confirm the clone worked
!ls ML-Assignment

# ---------------- Cell 03 ----------------
# --- Libraries ----------------------------------------------------------
import os
import json
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (train_test_split, RandomizedSearchCV, StratifiedKFold,
                                     cross_val_score, learning_curve)
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, f1_score, precision_score, recall_score,
                             classification_report, ConfusionMatrixDisplay,
                             roc_auc_score, roc_curve)
```

```
from sklearn.preprocessing import label_binarize


warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")

RANDOM_STATE = 42


# ---------- Cell 04 ----------

# --- Load the dataset ---------------------------------------------------

GITHUB_RAW_URL = "https://raw.githubusercontent.com/LoshithaAI/ML-Assignment/nissanka-hmdp-ms26930190/data/data.csv"

for path in ["data.csv", "data/data.csv", "../data/data.csv"]:

   if os.path.exists(path):

      DATA_PATH = path

      break

else:

   DATA_PATH = GITHUB_RAW_URL

df = pd.read_csv(DATA_PATH, sep=";")

df.columns = df.columns.str.strip()

print("Loaded from:", DATA_PATH)

print("Shape:", df.shape)


# ---------- Cell 05 ----------

# Display the first five records of the dataset

display(df.head())

print("Rows, columns:", df.shape)

print("Features:", df.shape[1] - 1, "| Target: Target")
```

```
# ---------- Cell 06 ----------
# Class distribution of the target
counts = df["Target"].value_counts()
print(pd.DataFrame({"count": counts, "percent": (counts / len(df) * 100).round(1)}))

plt.figure(figsize=(5, 3.5))
sns.countplot(data=df, x="Target", order=counts.index)
plt.title("Class distribution of the target")
plt.show()

# ---------- Cell 07 ----------
# Group the columns by type
cat_cols = ["Marital status", "Application mode", "Course", "Previous qualification", "Nacionality",
        "Mother's qualification", "Father's qualification", "Mother's occupation", "Father's occupation"]
binary_cols = [c for c in df.columns if c not in cat_cols + ["Target"] and df[c].nunique() <= 2]
num_cols = [c for c in df.columns if c not in cat_cols + binary_cols + ["Target", "Application order"]]

print("Categorical-code columns :", len(cat_cols))
print("Binary (0/1) columns     :", len(binary_cols))
print("Continuous columns       :", len(num_cols))
df[num_cols].describe().T.round(2)

# ---------- Cell 08 ----------
# data cleansing
print("Missing values (total):", int(df.isna().sum().sum()))
print("Duplicate rows       :", int(df.duplicated().sum()))
print("Non-numeric columns   :", list(df.select_dtypes(exclude="number").columns))
```

```
# ---------------- Cell 09 ----------------
# IQR rule: values outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR] are flagged as potential outliers
rows = []
for c in num_cols:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    n_out = int(((df[c] < q1 - 1.5 * iqr) | (df[c] > q3 + 1.5 * iqr)).sum())
    rows.append((c, n_out, round(n_out / len(df) * 100, 1)))
outliers = pd.DataFrame(rows, columns=["feature", "n_outliers", "percent"]).sort_values("n_outliers",
ascending=False)
display(outliers.head(8))


fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
for ax, c in zip(axes, ["Age at enrollment", "Admission grade", "Curricular units 1st sem (grade)"]):
    sns.boxplot(y=df[c], ax=ax)
    ax.set_title(c)
plt.tight_layout()
plt.show()



# ---------------- Cell 10 ----------------
# exploratory analysis
fig, axes = plt.subplots(1, 3, figsize=(14, 3.8))
for ax, c in zip(axes, ["Curricular units 1st sem (approved)",
                        "Curricular units 2nd sem (approved)",
                        "Admission grade"]):
    sns.boxplot(data=df, x="Target", y=c, order=["Dropout", "Enrolled", "Graduate"], ax=ax)
    ax.set_title(c, fontsize=10)
plt.tight_layout()
```

```
plt.show()


# ---------- Cell 11 ----------
# Creating Correlation matrix
plt.figure(figsize=(11, 8))
sns.heatmap(df[num_cols].corr(), cmap="coolwarm", center=0)
plt.title("Correlation matrix of continuous features")
plt.show()



# ---------- Cell 12 ----------
# encoding categorical features
X = pd.get_dummies(df.drop(columns="Target"), columns=cat_cols, drop_first=True)
y = df["Target"]
print("Features before encoding:", df.shape[1] - 1)
print("Features after encoding :", X.shape[1])



# ---------- Cell 13 ----------
#Train/test split
# 80/20 split, STRATIFIED so each part keeps the same class proportions.

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)

print("Train:", X_train.shape, "| Test:", X_test.shape)
print("Train class %:", (y_train.value_counts(normalize=True) * 100).round(1).to_dict())
print("Test  class %:", (y_test.value_counts(normalize=True) * 100).round(1).to_dict())
```

```
# ---------------- Cell 14 ----------------
# Model Implementation – Baseline model
baseline = RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE, n_jobs=-1)
baseline.fit(X_train, y_train)
pred_base = baseline.predict(X_test)

base_acc = accuracy_score(y_test, pred_base)
base_f1 = f1_score(y_test, pred_base, average="macro")
print(f"Baseline  accuracy = {base_acc:.3f} | macro-F1 = {base_f1:.3f}")

# ---------------- Cell 15 ----------------
# 5.2 Hyperparameter tuning (cross-validation on the training set only)
param_dist = {
   "n_estimators": [100, 200, 300, 500],
   "max_depth": [None, 8, 12, 16, 24],
   "min_samples_leaf": [1, 2, 4, 8],
   "max_features": ["sqrt", "log2", 0.3],
   "class_weight": [None, "balanced", "balanced_subsample"],
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
search = RandomizedSearchCV(
   RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
   param_distributions=param_dist, n_iter=20, cv=cv,
   scoring="f1_macro", random_state=RANDOM_STATE, n_jobs=1, return_train_score=True)
search.fit(X_train, y_train)

print("Best parameters  :", search.best_params_)
print(f"Best CV macro-F1 : {search.best_score_:.3f}")
```

```
cv_results = (pd.DataFrame(search.cv_results_)
          [["param_n_estimators", "param_max_depth", "param_min_samples_leaf",
"param_max_features",
            "param_class_weight", "mean_train_score", "mean_test_score"]]
          .sort_values("mean_test_score", ascending=False))
display(cv_results.round(3).head(8))


# ---------- Cell 16 ----------
# Overfitting check: train vs. validation score across all tried settings
plt.figure(figsize=(5.5, 4.5))
plt.scatter(cv_results["mean_train_score"], cv_results["mean_test_score"], alpha=0.7)
lo, hi = cv_results[["mean_test_score", "mean_train_score"]].min().min(), 1.0
plt.plot([lo, hi], [lo, hi], "k--", alpha=0.4, label="train = validation")
plt.xlabel("Mean training macro-F1"); plt.ylabel("Mean validation macro-F1")
plt.title("Train vs. validation score (20 settings)"); plt.legend()
plt.show()


# ---------- Cell 17 ----------
#final tuned model
best_model = search.best_estimator_
y_pred = best_model.predict(X_test)
y_proba = best_model.predict_proba(X_test)
classes = list(best_model.classes_)


test_acc = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred, average="macro")
print(f"Test accuracy : {test_acc:.3f}")
print(f"Test macro-F1 : {test_f1:.3f}")
```

```
print()
print(classification_report(y_test, y_pred, digits=3))



# ---------- Cell 18 ----------
#confusion matix
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axes[0], cmap="Greens", colorbar=False)
axes[0].set_title("Confusion matrix (counts)")
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, normalize="true", ax=axes[1],
                                        cmap="Greens", colorbar=False, values_format=".2f")
axes[1].set_title("Confusion matrix (row-normalised)")
plt.tight_layout()
plt.show()



# ---------- Cell 19 ----------
# One-vs-rest ROC curves and AUC
y_bin = label_binarize(y_test, classes=classes)
plt.figure(figsize=(6, 4.5))
for i, c in enumerate(classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
    plt.plot(fpr, tpr, label=f"{c} (AUC = {roc_auc_score(y_bin[:, i], y_proba[:, i]):.3f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.4)
plt.xlabel("False positive rate"); plt.ylabel("True positive rate")
plt.title("ROC curves (one-vs-rest)"); plt.legend()
plt.show()

macro_auc = roc_auc_score(y_test, y_proba, multi_class="ovr", average="macro")
```

```
print(f"Macro-average AUC (OvR): {macro_auc:.3f}")



# ---------- Cell 20 ----------
# Stability check: 5-fold CV of the final model on the TRAINING data
cv_acc = cross_val_score(best_model, X_train, y_train, cv=cv, scoring="accuracy")
cv_f1 = cross_val_score(best_model, X_train, y_train, cv=cv, scoring="f1_macro")
print(f"CV accuracy : {cv_acc.mean():.3f} +/- {cv_acc.std():.3f}")
print(f"CV macro-F1 : {cv_f1.mean():.3f} +/- {cv_f1.std():.3f}")

# Out-of-bag estimate -> free validation score built into Random Forest
oob_model = RandomForestClassifier(**{**search.best_params_, "oob_score": True,
                                      "random_state": RANDOM_STATE, "n_jobs": -1}).fit(X_train, y_train)
print(f"OOB accuracy: {oob_model.oob_score_:.3f}")



# ---------- Cell 21----------
# Summary table: baseline vs. tuned model
summary = pd.DataFrame({
    "Model": ["Random Forest (baseline)", "Random Forest (tuned)"],
    "Accuracy": [base_acc, test_acc],
    "Macro-F1": [base_f1, test_f1],
    "Macro-Precision": [precision_score(y_test, pred_base, average="macro"), precision_score(y_test,
y_pred, average="macro")],
    "Macro-Recall": [recall_score(y_test, pred_base, average="macro"), recall_score(y_test, y_pred,
average="macro")],
}).round(3)
summary
```

```
# ---------- Cell 22----------

# Model interpretation

# (a) Impurity-based importance (fast, but can favour features with many distinct values)

imp = pd.Series(best_model.feature_importances_,
index=X_train.columns).sort_values(ascending=False)

print("Top 15 features (impurity-based importance):")

display(imp.head(15).round(4))


plt.figure(figsize=(7, 5))

imp.head(15).sort_values().plot(kind="barh", color="tab:green")

plt.title("Random Forest feature importance (top 15)")

plt.xlabel("Mean decrease in impurity")

plt.tight_layout()

plt.show()



# ---------- Cell 23----------

# (b) Permutation importance on the TEST set (more reliable): how much does macro-F1 drop when one feature's values are shuffled?

top_feats = list(imp.head(25).index)

perm_model = RandomForestClassifier(**{**search.best_params_, "random_state": RANDOM_STATE,
"n_jobs": -1})

perm_model.fit(X_train, y_train)

base_score = f1_score(y_test, perm_model.predict(X_test), average="macro")


rng = np.random.RandomState(RANDOM_STATE)

drops = {}

for f in top_feats:

    scores = []

    for _ in range(3):
```

```
        Xp = X_test.copy()
        Xp[f] = rng.permutation(Xp[f].values)
        scores.append(f1_score(y_test, perm_model.predict(Xp), average="macro"))
    drops[f] = base_score - np.mean(scores)

perm = pd.Series(drops).sort_values(ascending=False)
plt.figure(figsize=(7, 5))
perm.head(12).sort_values().plot(kind="barh", color="tab:orange")
plt.title("Permutation importance (drop in test macro-F1)")
plt.xlabel("Decrease in macro-F1 when feature is shuffled")
plt.tight_layout()
plt.show()



# ---------- Cell 24 ----------
# Save results to be compared with the Logistic Regression results
os.makedirs("results", exist_ok=True)
results = {
    "model": "Random Forest",
    "best_params": {k: (None if v is None else (v if isinstance(v, (str, float)) else int(v)))
                    for k, v in search.best_params_.items()},
    "test_accuracy": round(float(test_acc), 4),
    "test_macro_f1": round(float(test_f1), 4),
    "test_macro_auc_ovr": round(float(macro_auc), 4),
    "cv_macro_f1_mean": round(float(cv_f1.mean()), 4),
    "per_class": classification_report(y_test, y_pred, output_dict=True, digits=4),
}
with open("results/random_forest_results.json", "w") as f:
    json.dump(results, f, indent=2)
```

```
print("Saved results/random_forest_results.json")



# ---------- Cell 25 ----------
# Reads both results files and compare
rows = []
for name, path in [("Logistic Regression", "results/logistic_regression_results.json"),
                   ("Random Forest", "results/random_forest_results.json")]:
    if os.path.exists(path):
        r = json.load(open(path))
        pc = r["per_class"]
        rows.append({"Model": name, "Accuracy": r["test_accuracy"], "Macro-F1": r["test_macro_f1"],
                     "Macro-AUC": r["test_macro_auc_ovr"],
                     "F1 Dropout": round(pc["Dropout"]["f1-score"], 3),
                     "F1 Enrolled": round(pc["Enrolled"]["f1-score"], 3),
                     "F1 Graduate": round(pc["Graduate"]["f1-score"], 3)})
    else:
        print(f"{path} not found yet - run the other notebook first.")
pd.DataFrame(rows)



# ---------- Cell 26----------
#Improvement experiments
tuned = dict(search.best_params_)
experiments = {}

def evaluate(name, params, Xtr, Xte):
    m = RandomForestClassifier(**{**params, "random_state": RANDOM_STATE, "n_jobs": -1}).fit(Xtr,
y_train)
```

```
    p = m.predict(Xte)
    experiments[name] = {"Accuracy": accuracy_score(y_test, p),
                         "Macro-F1": f1_score(y_test, p, average="macro"),
                         "Enrolled recall": recall_score(y_test, p, labels=["Enrolled"], average=None)[0]}
    return m

# E0: tuned model (reference)
evaluate("E0 Tuned model (reference)", tuned, X_train, X_test)

# E1: no class weights
evaluate("E1 Without class_weight", {**tuned, "class_weight": None}, X_train, X_test)

# E2: default hyperparameters (what tuning changed)
evaluate("E2 Default hyperparameters", {}, X_train, X_test)

# E3: raw integer codes instead of one-hot (trees can split on codes directly)
X_raw = df.drop(columns="Target")
Xr_train, Xr_test, _, _ = train_test_split(X_raw, y, test_size=0.2, stratify=y,
random_state=RANDOM_STATE)
evaluate("E3 Raw codes (no one-hot)", tuned, Xr_train, Xr_test)

# E4: feature selection - keep only the 30 most important features
top30 = list(imp.head(30).index)
evaluate("E4 Top-30 features only", tuned, X_train[top30], X_test[top30])

# E5: EARLY prediction - remove all 1st/2nd-semester performance features
early_cols = [c for c in X_train.columns if not c.startswith("Curricular units")]
evaluate("E5 Early prediction (no semester data)", tuned, X_train[early_cols], X_test[early_cols])
```

```
# E6: only information up to the 1st semester
sem1_cols = [c for c in X_train.columns if not c.startswith("Curricular units 2nd sem")]
evaluate("E6 Up to 1st semester only", tuned, X_train[sem1_cols], X_test[sem1_cols])

exp_table = pd.DataFrame(experiments).T.round(3)
exp_table



# ---------- Cell 27 ----------
#Bar chart comparing experiments
exp_table[["Accuracy", "Macro-F1", "Enrolled recall"]].plot(kind="bar", figsize=(11, 4.5))
plt.title("Improvement / ablation experiments (test set)")
plt.ylabel("Score"); plt.ylim(0, 1); plt.xticks(rotation=25, ha="right")
plt.legend(loc="lower left")
plt.tight_layout()
plt.show()



# ---------- Cell 28 ----------
# Effect of the NUMBER OF TREES
tree_counts = [10, 25, 50, 100, 200, 300, 500]
oob_scores, test_scores = [], []
for n in tree_counts:
    m = RandomForestClassifier(**{**tuned, "n_estimators": n, "oob_score": True,
                                  "random_state": RANDOM_STATE, "n_jobs": -1}).fit(X_train, y_train)
    oob_scores.append(m.oob_score_)
    test_scores.append(accuracy_score(y_test, m.predict(X_test)))
```

```
plt.figure(figsize=(6, 4))
plt.plot(tree_counts, oob_scores, marker="o", label="OOB accuracy")
plt.plot(tree_counts, test_scores, marker="o", label="Test accuracy")
plt.xlabel("Number of trees"); plt.ylabel("Accuracy"); plt.legend()
plt.title("Accuracy vs. number of trees")
plt.show()



# ---------- Cell 29 ----------
# Learning curve
sizes, tr_scores, va_scores = learning_curve(
    RandomForestClassifier(**{**tuned, "random_state": RANDOM_STATE, "n_jobs": -1}), X_train, y_train,
    train_sizes=np.linspace(0.1, 1.0, 6), cv=cv, scoring="f1_macro", n_jobs=1)

plt.figure(figsize=(6, 4))
plt.plot(sizes, tr_scores.mean(axis=1), marker="o", label="Training")
plt.plot(sizes, va_scores.mean(axis=1), marker="o", label="Validation")
plt.xlabel("Training set size"); plt.ylabel("macro-F1"); plt.legend()
plt.title("Learning curve")
plt.show()
```